كلية العلوم - أكاديـر
ⵜⴰⵖⵡⴰⵍⵜ ⵏ ⵜⵣⵔⴼⵜ - ⴰⴳⴰⴷⵉⵔ
FACULTÉ DES SCIENCES - AGADIR

Module: Systèmes à microprocesseur & à microcontrôleur

Prof. Said AGOUNAD
Laboratory of Metrology and Information Processing
Departement of Physics, Faculty of Sciences
Ibn Zohr University
Agadir, Morocco
email: s.agounad@uiz.ac.ma

16/02/2026

Objectifs de ce module

- Aider les étudiants à comprendre l'architecture des systèmes à microprocesseur et à microcontrôleur;
- Apprendre sur les modes et le langage de communication entre le processeur et ses périphériques;
- Préparer les étudient pour étudier les systèmes embarqués, systèmes de commandes/contrôles, systèmes intelligents, internet des objets connectés (IOT), etc;
- Permettre aux étudiants de toucher l'aspect pratique des systèmes à microprocesseur et à microcontrôleur à travers plusieurs exemples et projets.

Plan de ce module

- Partie I: Systèmes à microprocesseur
  - Chapitre I: Généralités sur les systèmes à microprocesseur
  - Chapitre II: Architecture et fonctionnement du microprocesseur Intel 8086
  - Chapiter III: Programmation en assembleur du µp 8086
  - Chapitre IV: Interfaces d'entrées/sorties et interruptions de µp 8086
- Partie II: Systèmes à microcontrôleur
  - Chapitre V: Microcontrôleurs : Arduino uno

Volume horaire : 47.5 H

- 11 Séances de cours (19.5 H)
  - Chap I »» 2 séances
  - Chap II »» 2 séances
  - Chaps. III »» 2 séances
  - Chaps IV »» 3 séances
  - Chaps IV »» 2 séances
- 10 Séances de TD (17.5 H)
  - Chaps I, II, III, IV, V »» 2 séances pour chaque chapitre
- 3 Séances de TP (10.5 H)

Évaluation 20/20 (100 %)

- Cours + TD »» 75%
  - Examen final »» 60 %
  - Notes des quizs et tests au niveau de la plateforme (ecourse-fs.uiz.ac.ma) »» 15 %
- Colle de TP »» 25 %



Chapitre I : Généralités sur les systèmes à microprocesseur

Prof. Said AGOUNAD
Laboratory of Metrology and Information Processing
Departement of Physics, Faculty of Sciences
Ibn Zohr University
Agadir, Morocco
email: s.agounad@uiz.ac.ma

16/02/2026

Définition :

Un **système à microprocesseur** est un système électronique programmable équipé d'éléments nécessaires pour réaliser une fonction donnée. Parmi ces éléments, on trouve le microprocesseur, l'horloge, les mémoires, les interfaces entrées/sorties et les bus, etc.

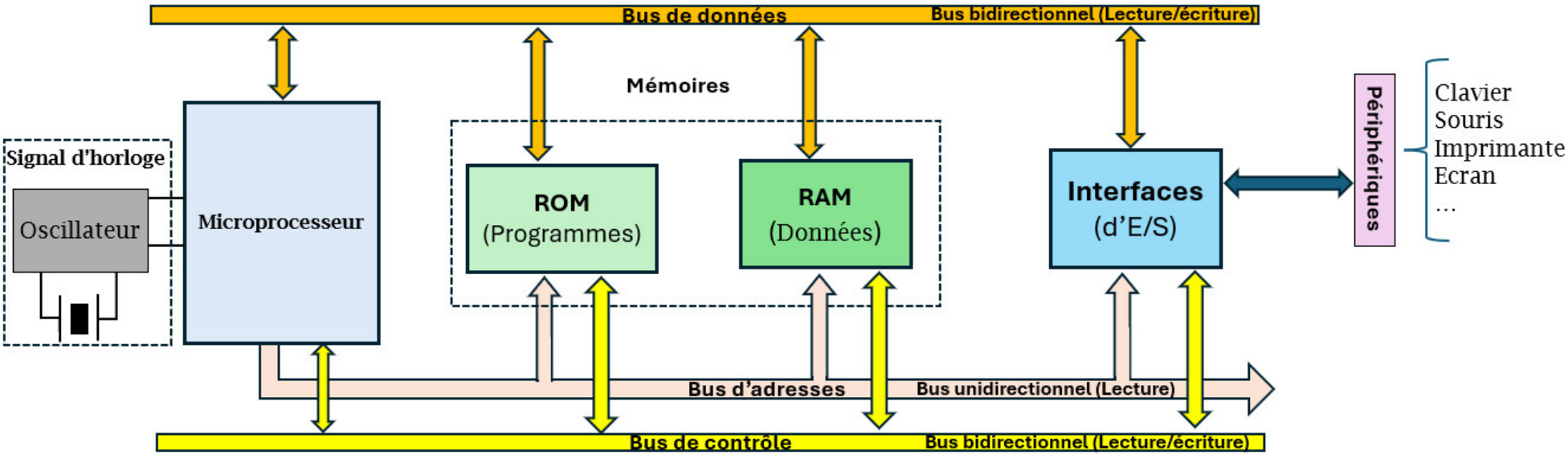


Figure: Système à microprocesseur.

Domaines d'applications

Les systèmes à microprocesseur sont omniprésentes dans :







Industrie et Automatisation

Électronique grand public

Domaines d'applications des systèmes à microprocesseurs

Appareils ménagers (domotique)

Automobile et transport

Télécommunications et Aérospatial

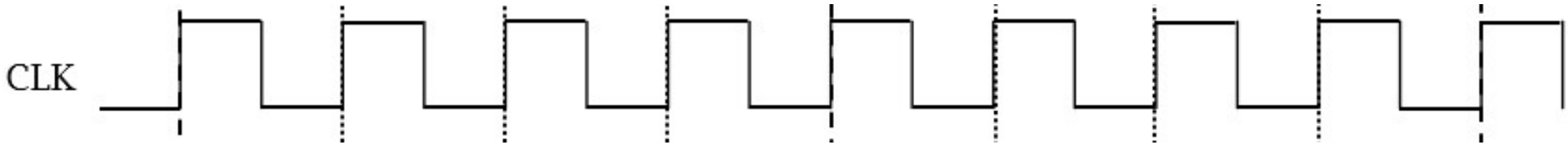
Santé et Instrumentation







▷ **Définition** : **Signal d'horloge** ou tout simplement l'horloge est un signal périodique qui synchronise toutes les opérations internes du microprocesseur (récupération d'instructions, traitement des données). Il est généré par oscillateur électrique à quartz



▷ La fréquence (vitesse du microprocesseur) du signal d'horloge, mesurée en Hz ou plus souvent en GHz, indique le nombre de cycles (impulsions) que le microprocesseur exécute par seconde.

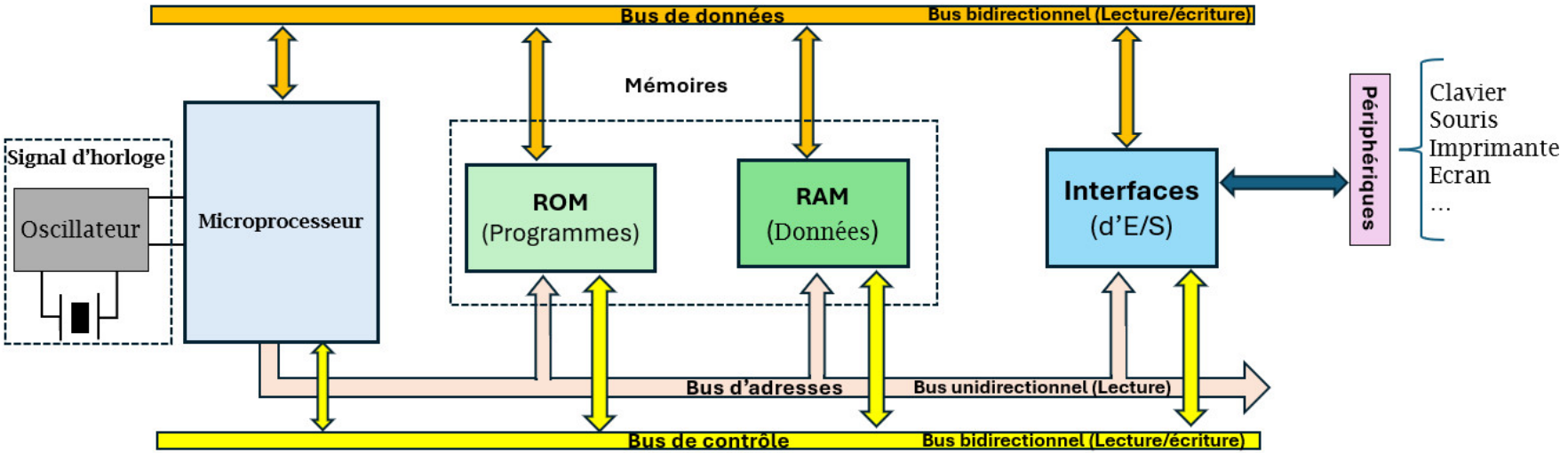
▷ **Exemple** : Pour une horloge de fréquence 3 GHz, le microprocesseur effectue 3 milliards de cycles par seconde.

▷ La durée d'exécution d'une instruction dépend de la période de l'horloge et le nombre de cycles nécessaires à leur exécution.

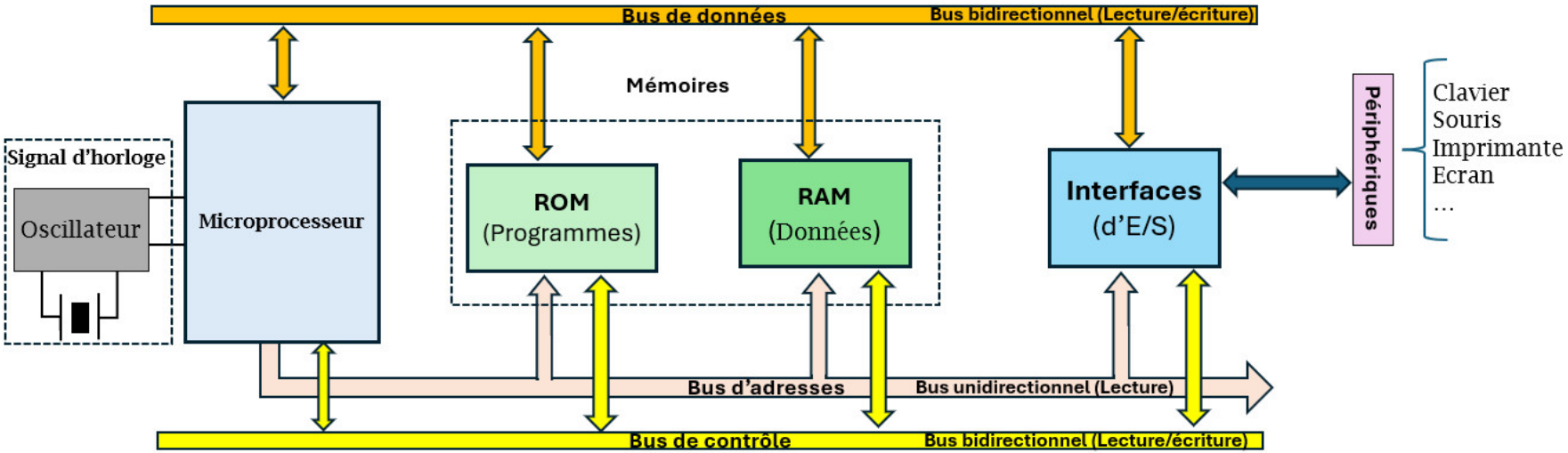
▷ **Exemple :** La durée d'exécution de l'instruction qui s'exécute en trois cycles d'horloge de fréquence 5 MHz (période $T = 1/f = 0,2$ us) est : $3 \times 0,2 = 0,6$ us.

▷ **Définition:** Un bus est constitué de fils électriques qui servent à transporter des informations entre deux éléments ou plus.

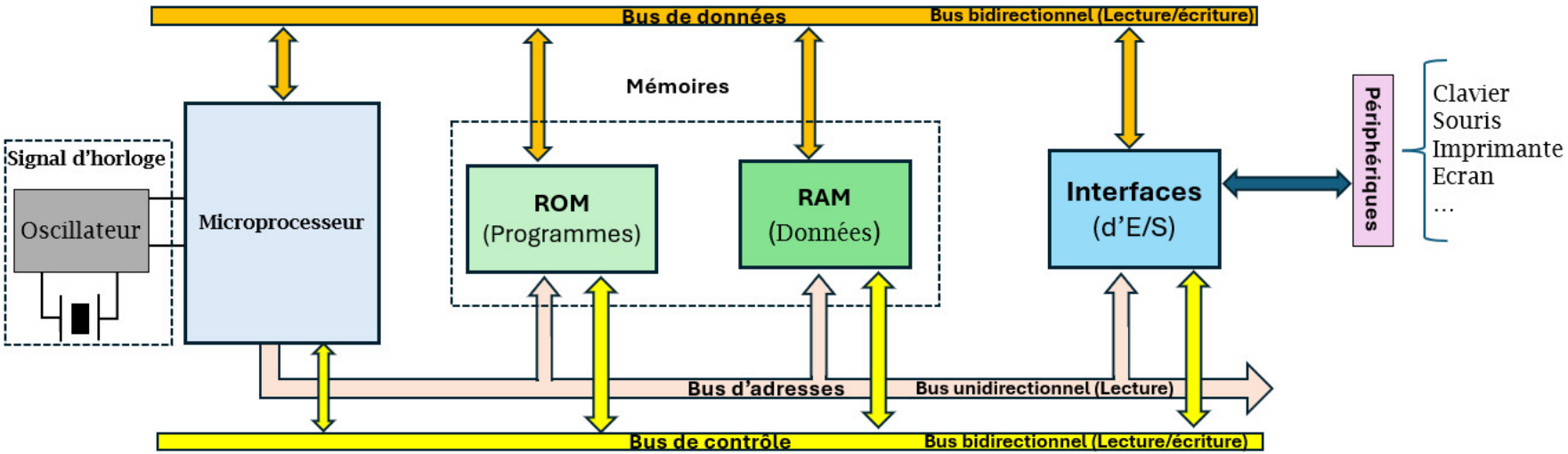
▷ Dans le cas d'un système à microprocesseur, la communication et l'échange des informations entre les différents éléments de ce système sont assurés par le biais de trois bus :



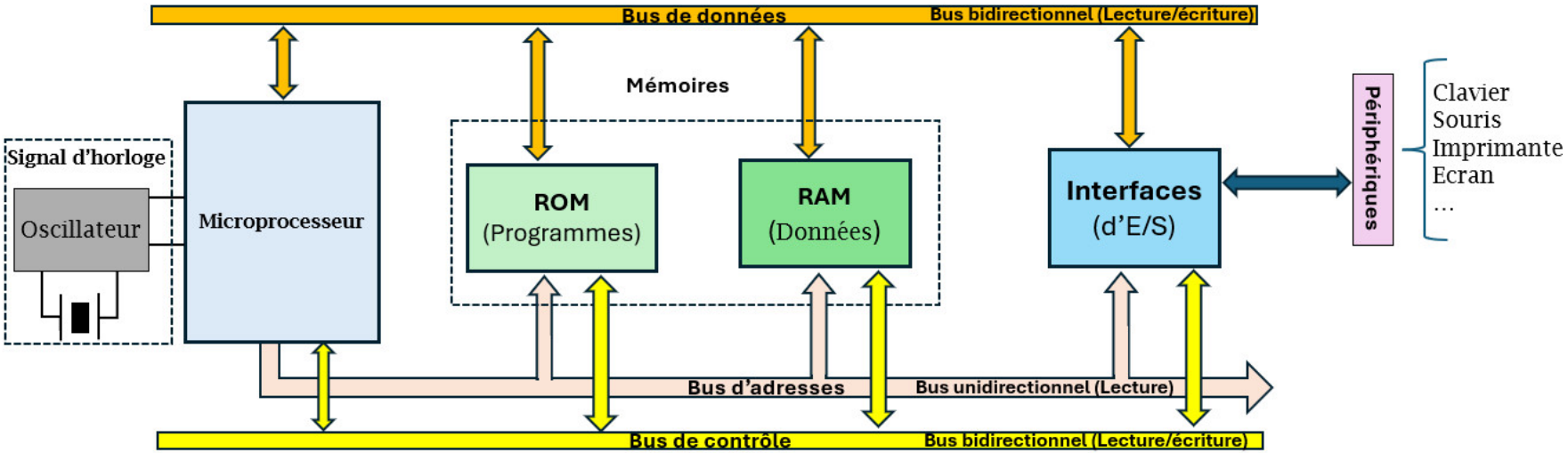
- bus de données :
  - bidirectionnel; transporte les données et les instructions;
  - Plusieurs organes sont reliés à ce bus, mais un organe ne peut y expédier des données vers un autre organe qu'une fois reçu le signal correspondant émis par la logique de décodage et de séquencement en fonction de l'instruction en cours;
  - **Exemple** : Un microprocesseur 8 bits (Ex. Motorola 6809) possède un bus de données qui contient 8 fils électriques, notés D0 à D7. Il peut transmettre 8 bits (1 octet) à la fois.



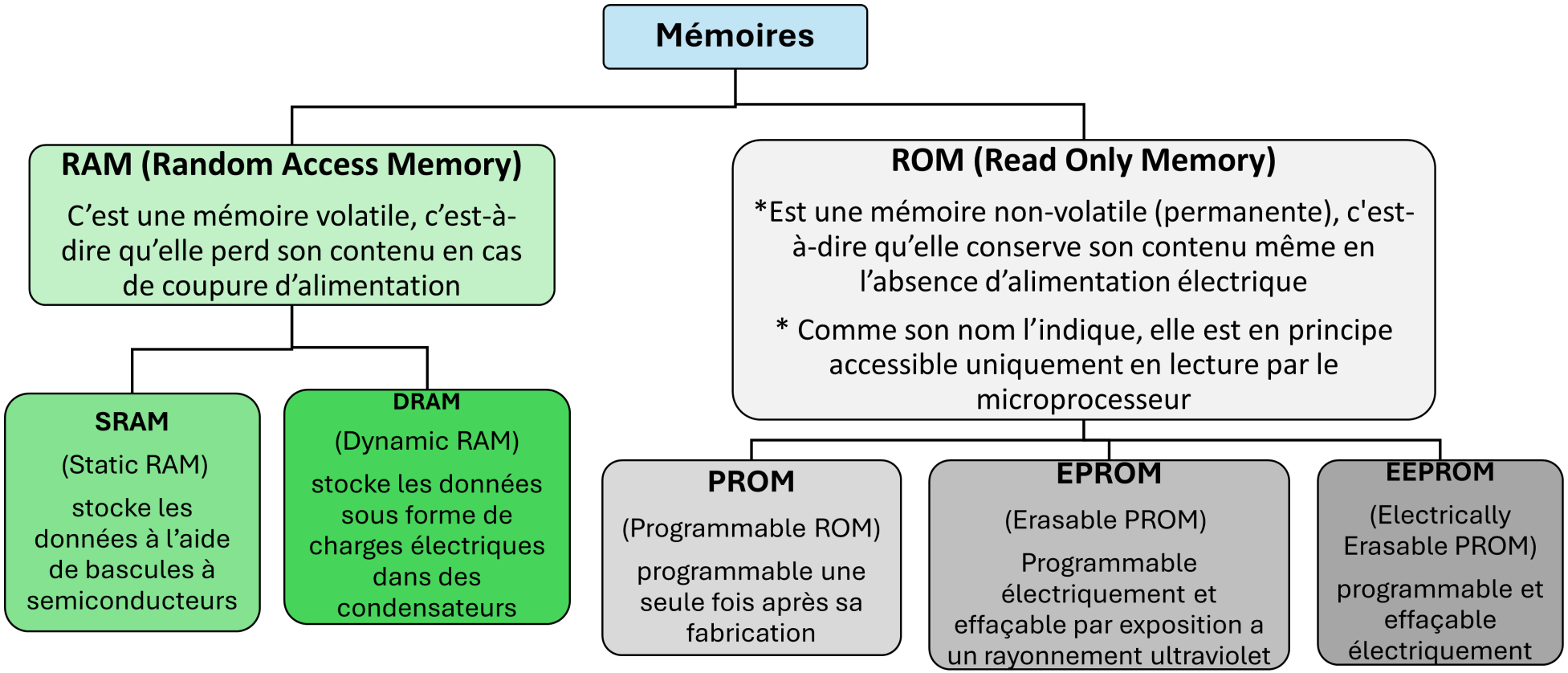
- bus de contrôle:
  - bidirectionnel
  - synchroniser le système: Horloge;
  - réaliser l'initialisation de l'unité centrale: Reset;
  - contrôler les échanges (lecture/écriture): R/W (read/write);
  - provoquer des exécutions de programmes particuliers (interruptions): IRQ.



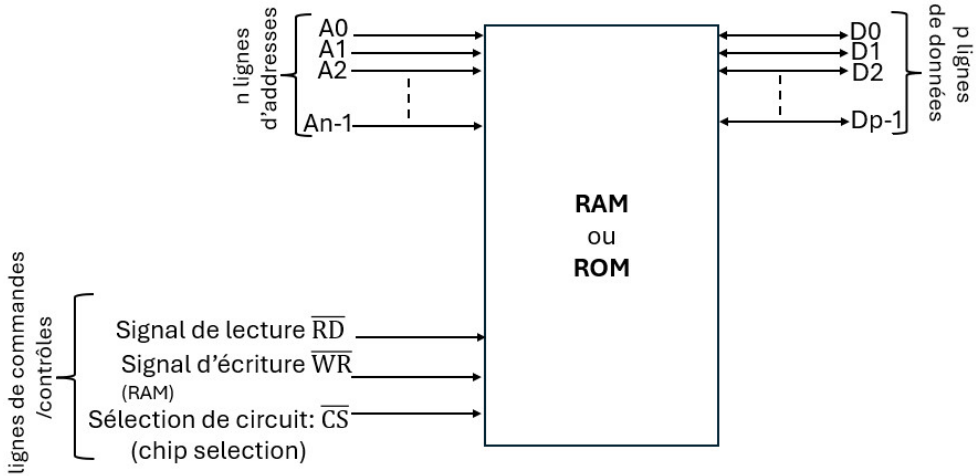
- bus d'adresses :
  - unidirectionnel; transporte des adresses;
  - Le nombre de bits d'adresses permet de déterminer le nombre de cases mémoires que le microprocesseur peut adresser : c-à-d **l'espace adressable**.
  - **Exemple** : Pour un bus d'adresse de 16 bits, notés A0 à A15, le microprocesseur peut accéder à 2^{16} octets (=65 536 octets =64 Kilo octets; noté que 1 kilo-octet=1024 octets) si le bus de données est de 8 bits.



▷ **Définition**: Une mémoire est un dispositif électronique qui sert à stocker des données. On distingue généralement entre deux types de mémoires: Mémoires vives (RAM) et les Mémoires mortes (ROM)



▷ Schéma fonctionnel d'une mémoire:



- n broches d'adresses permettant de sélectionner une case mémoire
- p broches de données permettant à chaque case mémoire de fournir ou recevoir une information codée sur p bits.
- 3 broches de commande/contrôle:
  - $\overline{RD}$ (Read): active la lecture et place sur les lignes de sortie la donnée contenue à l'adresse sélectionnée.
  - $\overline{WR}$ (Write) : autorise l'écriture d'une donnée à l'adresse choisie.
  - $\overline{CS}$ (Chip Select) : permet de sélectionner le circuit mémoire concerné.

La barre qui se trouve sur des broches comme $\overline{RD}$ signifie que cette broche est activée à l'état bas.

▷ **Définition**: La capacité d'une mémoire (taille), notée C, est le nombre d'informations (quantité) que la mémoire peut mémoriser. Elle est définie par :

$$C = 2^n xp \quad bits$$

n: représente le nombre de lignes d'adresses

p: représente le nombre de bits qu'une case mémoire peut stocker, c.à.d., la taille d'une case mémoire qui peut être 8, 16, 32, 64 bits ou plus

- **Bit** (0 ou 1): est un élément de base pour la représentation des info.
- **Octet** : est une suite de 8 bits codant une information. Il y a des multiples d'un octet tels que :
  - Kilo-octet (Ko): 1 Ko =1024 octets =2^{10} octets
  - Méga-octet (Mo): 1 Mo =1024 Ko =2^{20} octets
  - Géga-octet (Go): 1 Go =1024 Mo =2^{30} octets
  - Téra-octet (To): 1 To =1024 Go =2^{40} octets

▷ **Exemple:** Trouver la taille d'une mémoire de 8 lignes d'adresse et celle d'une mémoire de 16 lignes

▷**Réponse :** 8 bits d'adresses permettent d'adresser $2^8 = 256$ cases mémoire, c.à.d., la taille de mémoire est 256 octets (si $p = 8$). 16 bits d'adresses permettent d'adresser $2^{16} = 65536$ cases mémoire, c.à.d., la taille de mémoire est 64 Ko (si $p = 8$).

▷ **Interfaçage microprocesseur/mémoire**:

- Un microprocesseur peut être relié à un ou plusieurs mémoires en même temps qui sont reliées en permanence sur le bus de données.

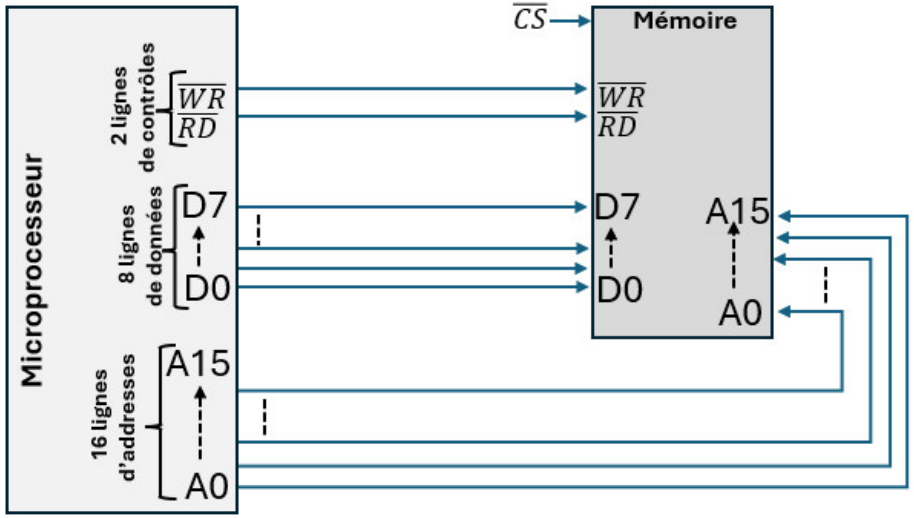
- La sélection d'une seule mémoire pour la lecture ou l'écriture se fait à l'aide de décodeur d'adresse qui commande les broches $\overline{CS}$ des différentes mémoires.

  - Lorsque une broche $\overline{CS}$ est active (à l'état bas), le boîtier de mémoire correspondant est sélectionné pour la lecture ou l'écriture.

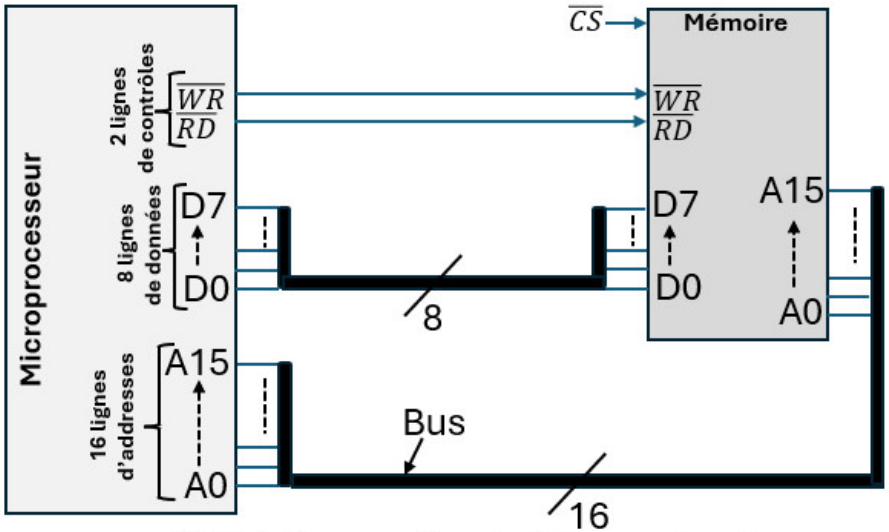
  - Lorsque $\overline{CS}$ est inactive (à l'état haut), la mémoire est mise hors service : ses broches de données (D0 à Dp-1) passent en haute impédance, donc la mémoire est déconnectée du bus de données du microprocesseur.

▷ **Interfaçage microprocesseur/mémoire**:

▷**Exemple 1**: Interfaçage microprocesseur/1 boîtier mémoire d'une capacité de 64 ko sur un bus d'adresse de 16 bits.



(a) Schéma détaillé de l'interfaçage microprocesseur–mémoire

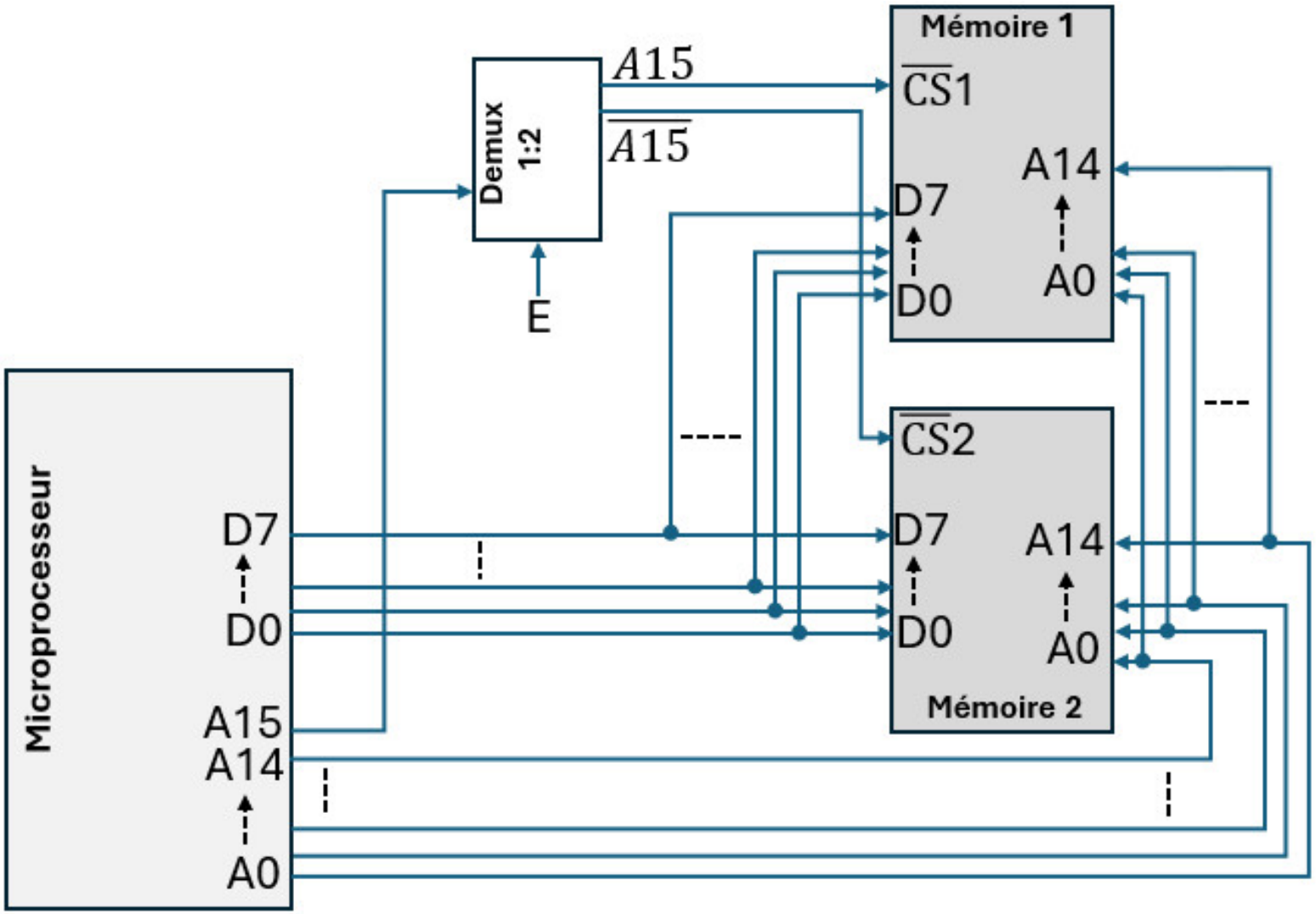


(b) Schéma condensée (plus pratique)

Figure: Interfaçage microprocesseur/1 boîtier mémoire.

▷ **Interfaçage microprocesseur/mémoire**:

▷**Exemple 2**: Interfaçage microprocesseur/2 boîtiers mémoire d'une capacité de 32 Ko chacune (15 lignes d'adresses) sur un bus d'adresse de 16 bits.



▷ **Interfaçage microprocesseur/mémoire**:

▷**Exemple 3** : Interfaçage microprocesseur/4 boîtiers mémoire d'une capacité de 256 octets chacune (8 lignes d'adresses) sur un bus d'adresse de 10 bits.

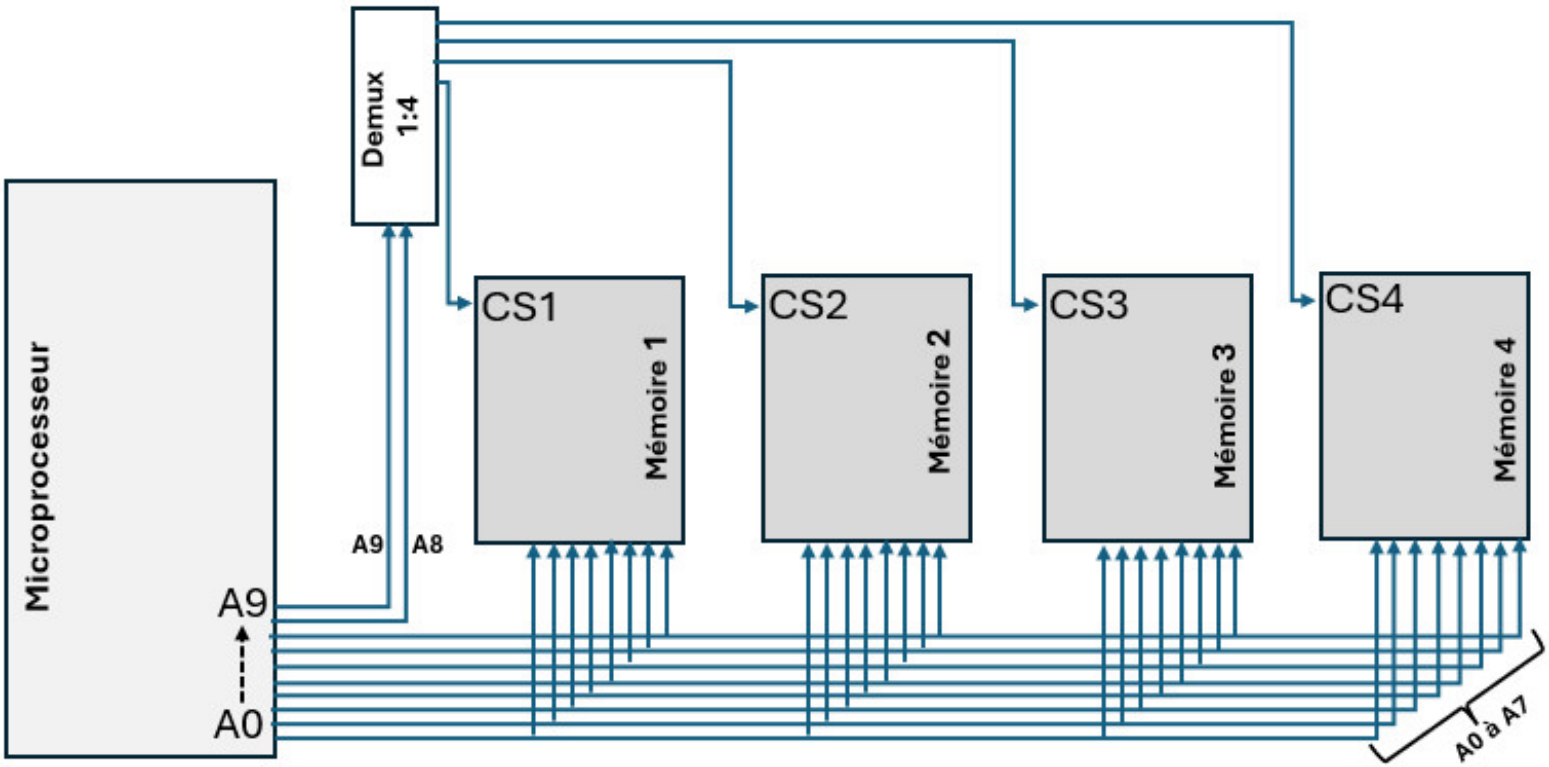


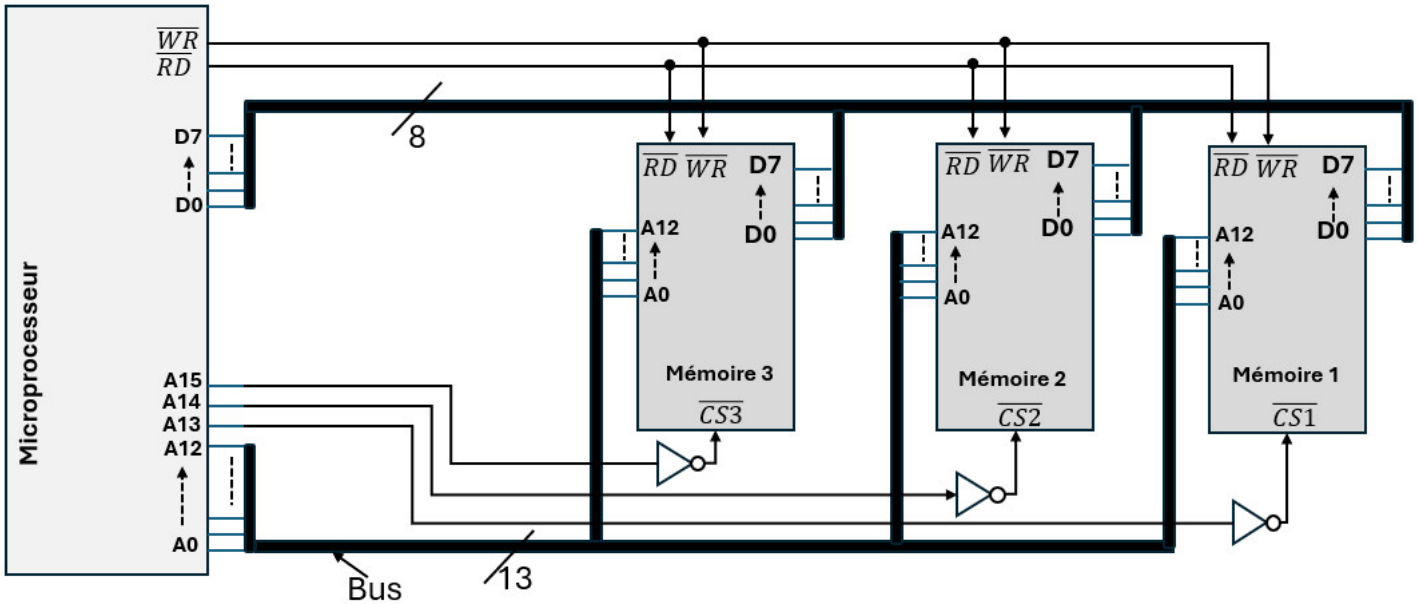
Figure: Interfaçage microprocesseur/4 boîtiers mémoire.

▷ **Décodage d'adresses** est une opération qui permet d'allouer à chaque mémoire (ou périphérique) une zone de l'espace adressable.
▷La capacité d'adressage du microprocesseur est souvent plus grande que la capacité des mémoires implantées. Donc, il est possible d'ajouter d'autres boîtiers mémoire afin d'exploiter tout l'espace adressable.
Exemple: Connexion de 3 boîtiers de 8 Ko sur un bus d'adresse de 16 bits.

▷ **Décodage d'adresses** est une opération qui permet d'allouer à chaque mémoire (ou périphérique) une zone de l'espace adressable.

▷La capacité d'adressage du microprocesseur est souvent plus grande que la capacité des mémoires implantées. Donc, il est possible d'ajouter d'autres boîtiers mémoire afin d'exploiter tout l'espace adressable.

Exemple: Connexion de 3 boîtiers de 8 Ko sur un bus d'adresse de 16 bits.



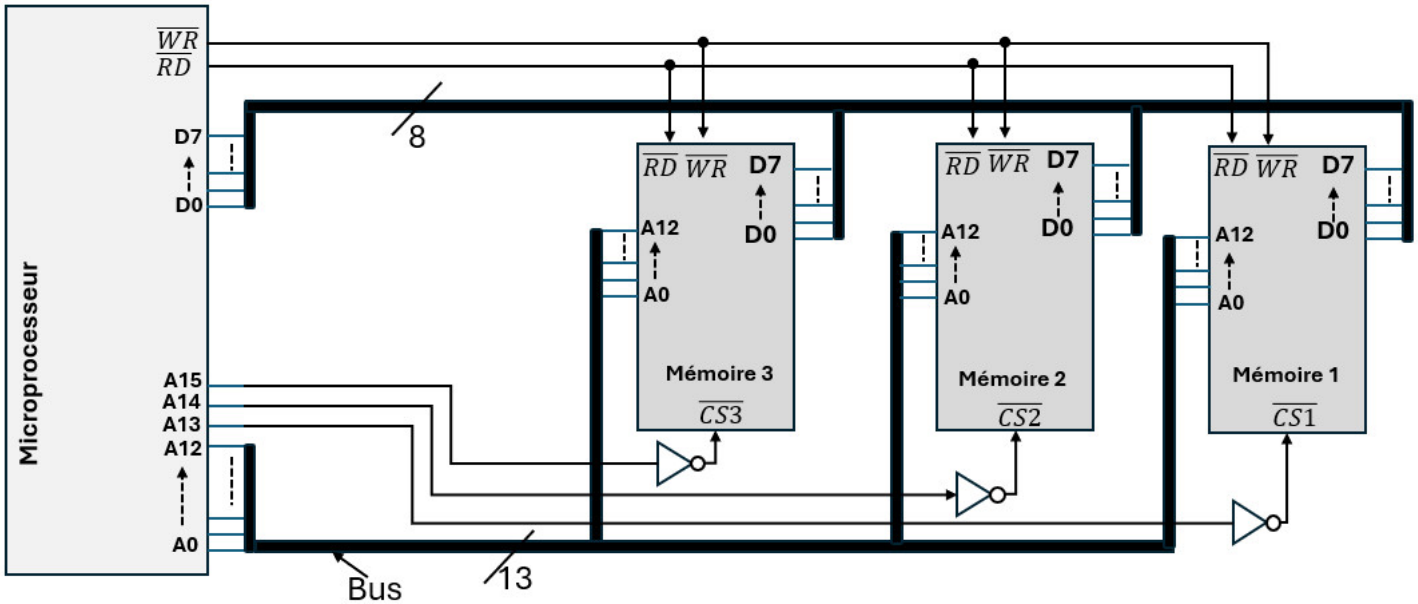
▷ Les lignes d'adresses A13, A14 et A15 sont utilisées pour sélectionner un seul boîtier mémoire à la fois à l'aide du circuit de décodage d'adresses.

▷ Ils peuvent prendre des valeurs binaires de 000 à 111, soit 8 combinaisons possibles, ce qui permet de gérer jusqu'à 8 boîtiers mémoires distincts. ▷

▷ **Décodage d'adresses** est une opération qui permet d'allouer à chaque mémoire (ou périphérique) une zone de l'espace adressable.

▷La capacité d'adressage du microprocesseur est souvent plus grande que la capacité des mémoires implantées. Donc, il est possible d'ajouter d'autres boîtiers mémoire afin d'exploiter tout l'espace adressable.

Exemple: Connexion de 3 boîtiers de 8 Ko sur un bus d'adresse de 16 bits.

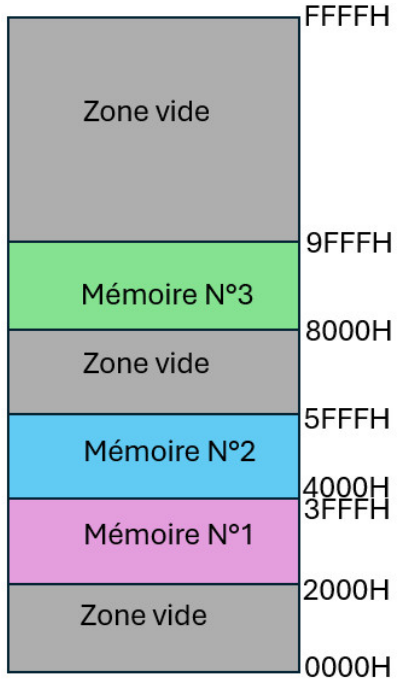


▷ Quelle est la plage d'adresse occupée par chacune de ces mémoires?

Réponse: ▷Pour atteindre la mémoire N°1, il faut mettre à 1 le bit A13 et à 0 les bits A14 et A15. La plage d'adresses occupée par cette mémoire est

A15	A14	A13	A12	....	A2	A0		A15	A14	A13	A12	....	A2	A0	
0	0	1	0	....	0	0	→	0	0	1	1	....	1	1	(binaries)
2000								3FFF							(hexadecimal)

▷On en déduit la cartographie ou mapping de la mémoire visible par le microprocesseur:



▷ De même, pour la mémoire N°2, on doit avoir A13 = 0, A14 = 1 et A15 = 0 d'où la plage d'adresses occupée cette mémoire :

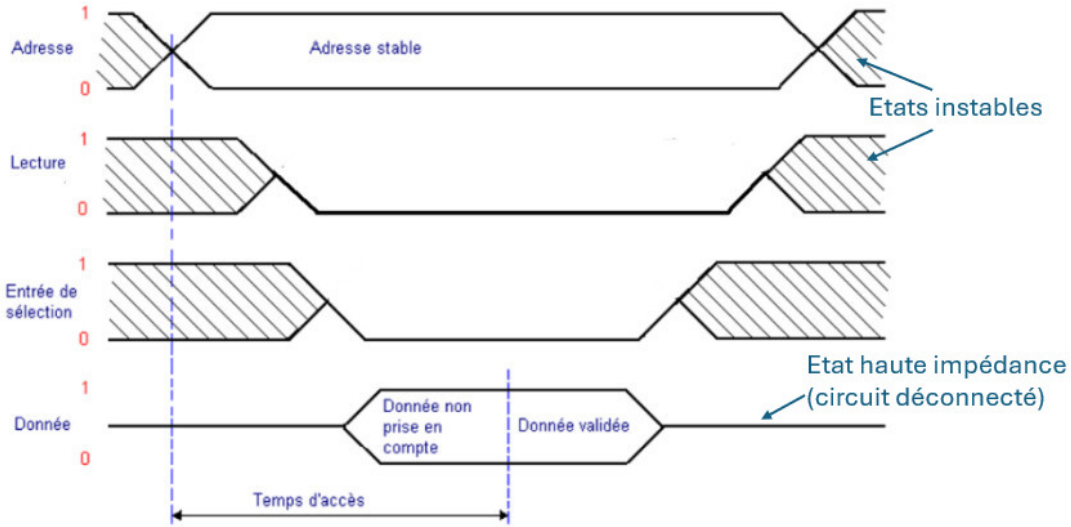
A15	A14	A13	A12	....	A2	A0		A15	A14	A13	A12	....	A2	A0	
0	1	0	0	....	0	0	→	0	1	0	1	....	1	1	(binaries)
4000								5FFF							(hexadecimal)

▷ Pour la mémoire N°3, on doit avoir A13 = 0, A14 = 0 et A15 = 1 d'où la plage d'adresses occupée cette mémoire :

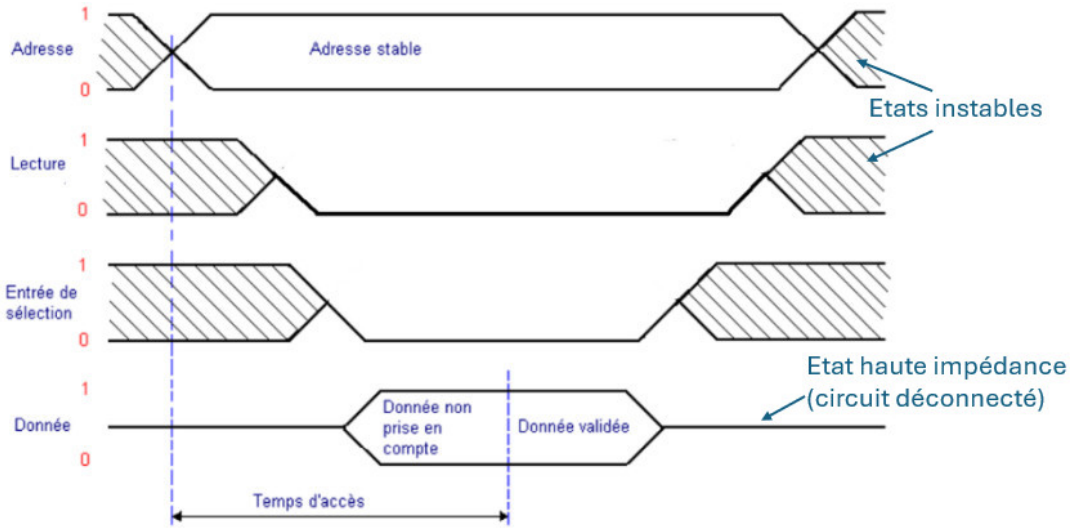
A15	A14	A13	A12	....	A2	A0		A15	A14	A13	A12	....	A2	A0	
1	0	0	0	....	0	0	→	1	0	0	1	....	1	1	(binaries)
8000								9FFF							(hexadecimal)

▷ **Chronogramme de lecture en mémoire**: Les étapes de lecture d'une case mémoire sont:

- Présentation de l'adresse de la case mémoire (donnée) que le microprocesseur veut lire;
- Mettre l'entrée de lecture $\overline{RD}$ à l'état actif "dans ce cas c'est 0";
- Sélection du boîtier de la mémoire, en appliquant sur les entrées de sélection, $\overline{CS}$, un niveau actif, "dans ce cas c'est 0".
- Mettre la valeur à lire, pendant une durée donnée, sur les bus de données.



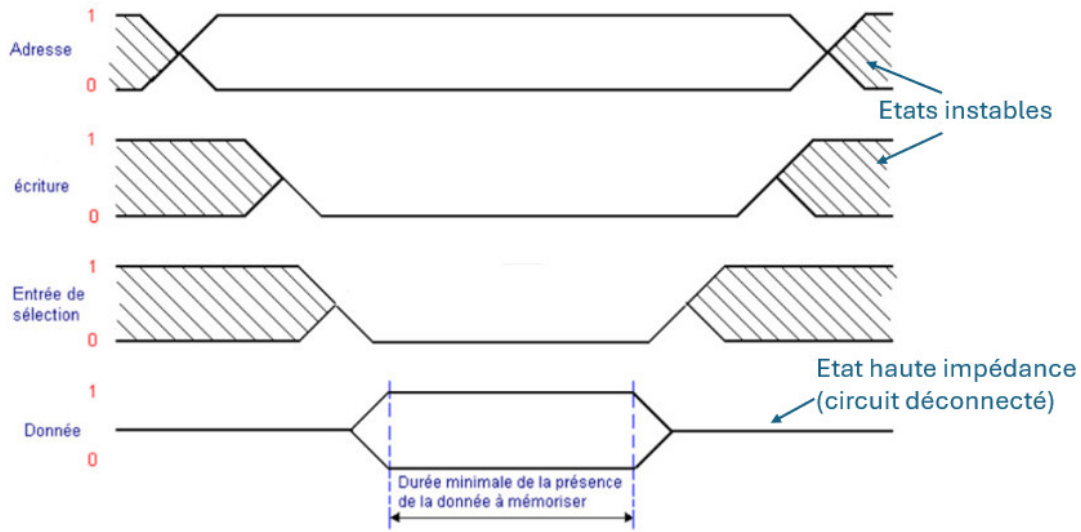
- Une fois, les entrées de sélection sont activées, le bus de données



- Une fois, les entrées de sélection sont activées, le bus de données bascule de l'état haute impédance à l'état actif, mais ne donne pas cependant la donnée correspondante à l'adresse désirée. En effet, le temps de propagation de l'état des entrées de lecture/écriture et d'adresse est plus grande que celui des entrées de sélection.
- temps d'accès au mémoire est une caractéristique de la mémoire qui définit la durée qui s'écoule entre le moment où une adresse est appliquée sur le bus d'adresses et le moment où la donnée correspondante devient disponible et stable sur le bus de données.

▷ **Chronogramme de l'écriture en mémoire**: Les étapes de l'écriture d'une case mémoire sont:

- Le µp présente l'adresse de la case mémoire où il veut mémoriser une donnée;
- Ensuite, il active l'entrée d'écriture à l'état actif, dans ce cas c'est 0, $\overline{WR}$;
- Sélection du boîtier de la mémoire, en appliquant sur les entrées de sélection, $\overline{CS}$, un niveau actif, "dans ce cas c'est 0".
- Enfin il applique la valeur à mémoriser, pendant un temps au moins égal au temps d'accès de la mémoire, sur les bus de données.



▷ **Historique des microprocesseurs** : Le premier microprocesseur a été inventé en 1969 par les physiciens de Intel : **Marcian Hoff** (ingénieur docteur en électronique américain) et **Federico Faggin** (physicien et inventeur italien).

Date	Invention	Fabriquant	Horloge	Géométrie	Nombre de transistors	Loi de Moore
1971	Lancement du premier microprocesseur commercial, 4004 (4 bits)	Intel	740 kHz	10 µm	2 250	2 250
1974	6800 (8 bits)	Motorola	2 MHz	-	4 100	6 750
1974	8080 (8 bits)	Intel	2 MHz	6 µm	6 000	6 750
1978	8086 (16 bits)	Intel	5 MHz	3 µm	29 000	27 000
1979	6809 (8 bits)	Motorola	1 MHz	5 µm	40 000	36 000
1984	68020 (32 bits)	Motorola	16 MHz	2 µm	190 000	216 000
1985	80386 (32 bits)	Intel	16 MHz	1,5 µm	275 000	288 000
1991	R-4000 (64 bits)	MIPS	100 MHz	0,8 µm	1,35 millions	2,3 millions
1993	Pentium (64 bits)	Intel	60 MHz	0,8 µm	3,1 millions	4,6 millions
2006	Core 2	Intel	3 GHz	65 nm	291 millions	442,5 millions
2007	Phenom	AMD	2,6 GHz	65 nm	463 millions	590 millions
2008	Core i7	Intel	3,2 GHz	45 nm	730 millions	885 millions
2017	Ryzen 1000 (nbr. de cœurs 4-16)	AMD	3,2-4,1 GHz	14 nm	4 800 millions	-

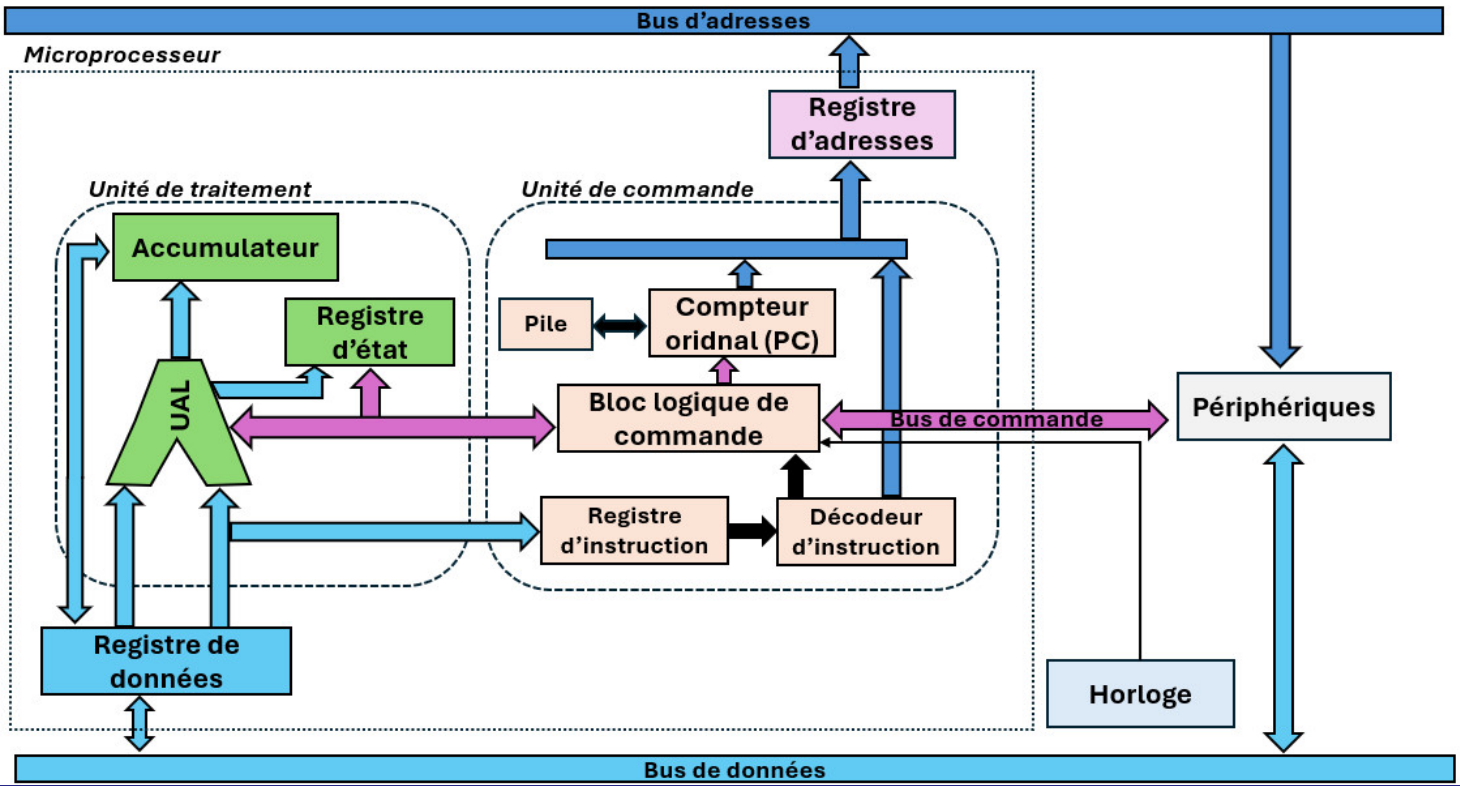
Date	Invention	Fabriquant	Horloge	Géométrie	Nombre de transistors	Loi de Moore
1971	Lancement du premier microprocesseur commercial, 4004 (4 bits)	Intel	740 kHz	10 µm	2 250	2 250
1974	6800 (8 bits)	Motorola	2 MHz	-	4 100	6 750
1974	8080 (8 bits)	Intel	2 MHz	6 µm	6 000	6 750
1978	8086 (16 bits)	Intel	5 MHz	3 µm	29 000	27 000
1979	6809 (8 bits)	Motorola	1 MHz	5 µm	40 000	36 000
1984	68020 (32 bits)	Motorola	16 MHz	2 µm	190 000	216 000
1985	80386 (32 bits)	Intel	16 MHz	1,5 µm	275 000	288 000
1991	R-4000 (64 bits)	MIPS	100 MHz	0,8 µm	1,35 millions	2,3 millions
1993	Pentium (64 bits)	Intel	60 MHz	0,8 µm	3,1 millions	4,6 millions
2006	Core 2	Intel	3 GHz	65 nm	291 millions	442,5 millions
2007	Phenom	AMD	2,6 GHz	65 nm	463 millions	590 millions
2008	Core i7	Intel	3,2 GHz	45 nm	730 millions	885 millions
2017	Ryzen 1000 (nbr. de cœurs 4-16)	AMD	3,2-4,1 GHz	14 nm	4 800 millions	-
2017	Core i9 (nbr. de coeurs 8 à 18)	Intel	2,60 GHz à 4,30 GHz	14 nm	-	-
2021	Rocket Lake	Intel	1,3 GHz à 5,3 GHz	14 nm	-	-
2023	Meteor Lake	Intel	1,3 GHz à 5,3 GHz	14 nm	-	-

Table: Quelques dates importantes dans les l'évolution des microprocesseurs.

▷ **Architecture de base d'un microprocesseur**: un microprocesseur est construit autour de trois éléments principaux:

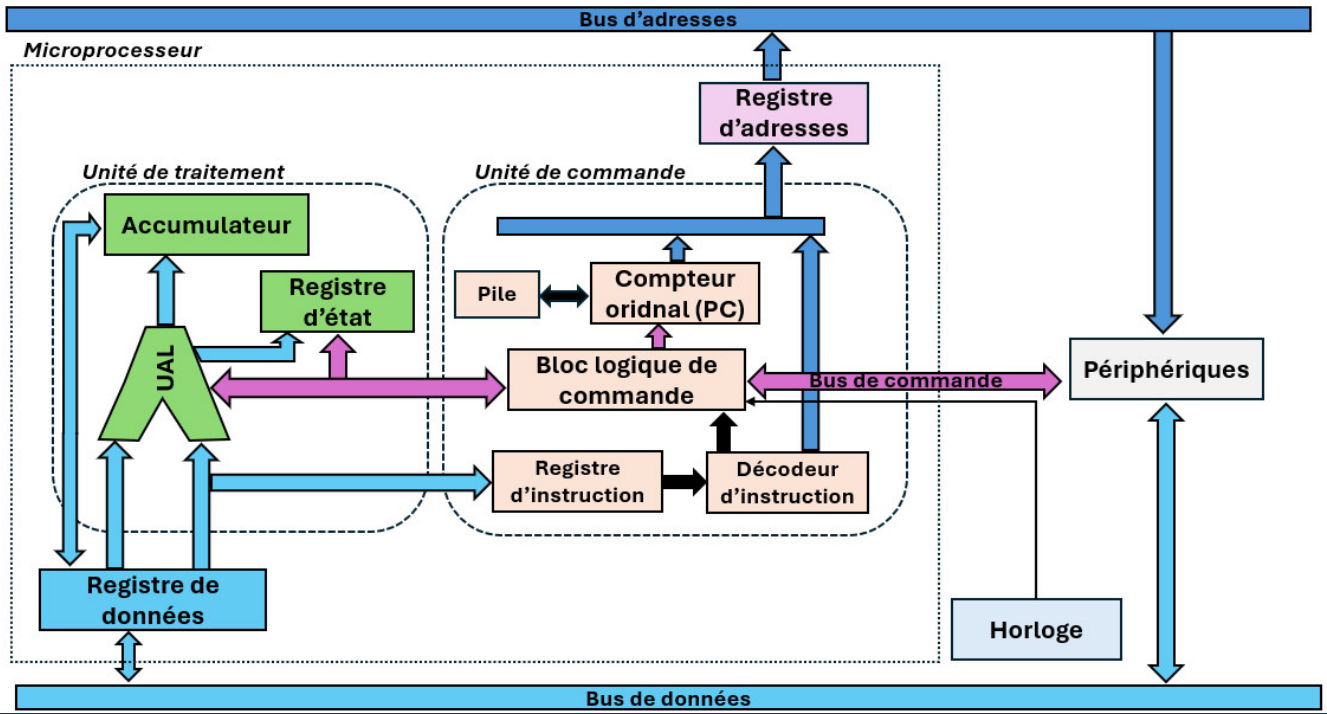
- unité de traitement;
- unité de commande;
- registres de données et d'adresses.

Ces trois éléments sont reliés entre eux par des bus internes permettant les échanges d'informations.



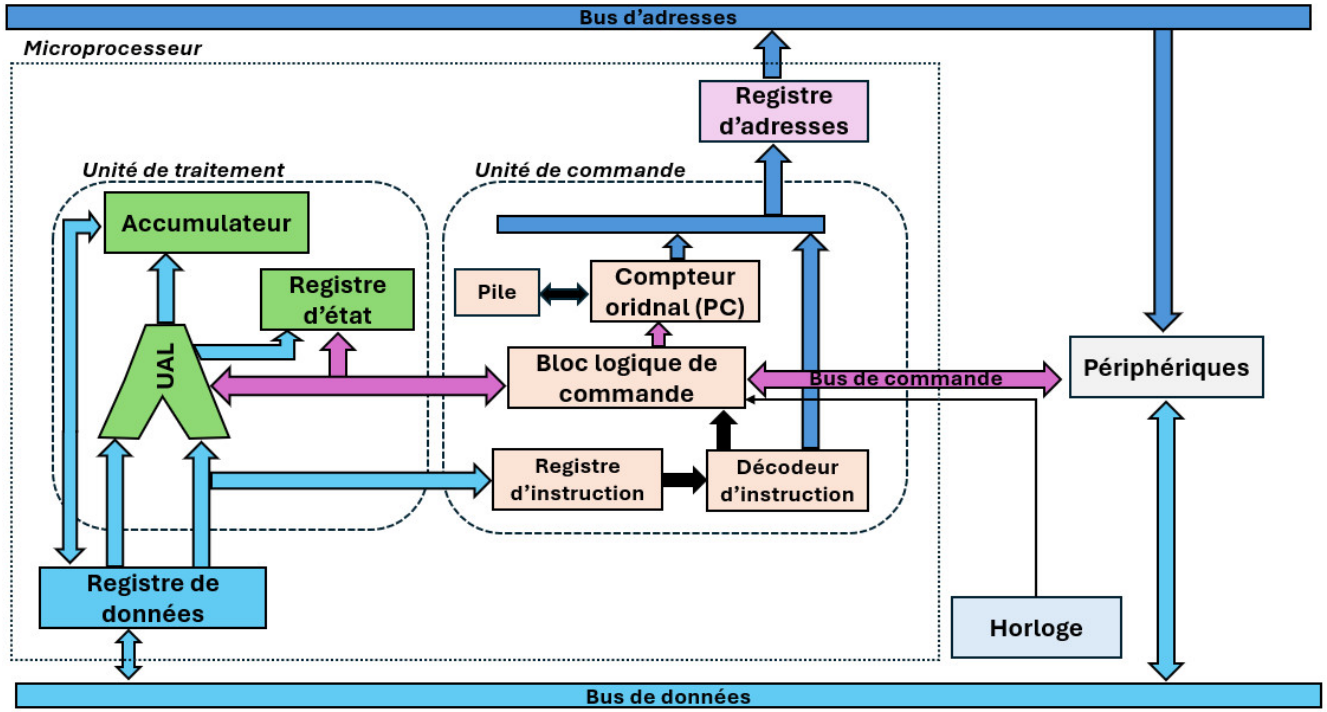
▷ **Architecture de base d'un microprocesseur** : L'unité de traitement représente le cœur du microprocesseur, dont le rôle est le traitement des données. Elle est constitué des éléments suivants :

- Unité Arithmétique et Logique (UAL, ou ALU de l'anglais "arithmetic and logic unit") est un circuit qui réalise les opérations arithmétiques (Addition, soustraction...) et logiques (ET, OU...). Les données à traiter arrivent dans l'UAL par l'intermédiaire du registre de données. Le résultat de l'opération exécutée sont inscrits dans l'accumulateur.



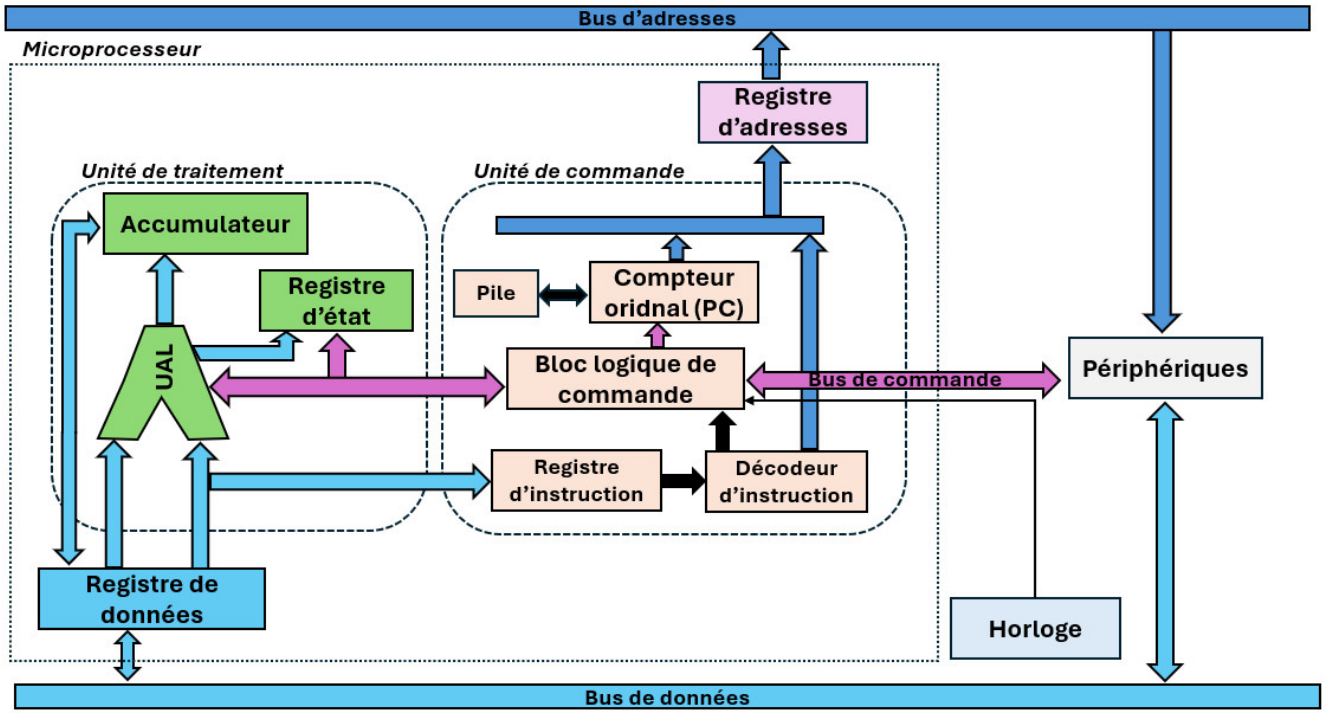
▷ **Architecture de base d'un microprocesseur** : L'unité de traitement représente le cœur du microprocesseur, dont le rôle est le traitement des données. Elle est constitué des éléments suivants :

- Accumulateur est registre de travail qui serve à stocker un opérande au début d'une opération arithmétique et le résultat à la fin de l'opération.



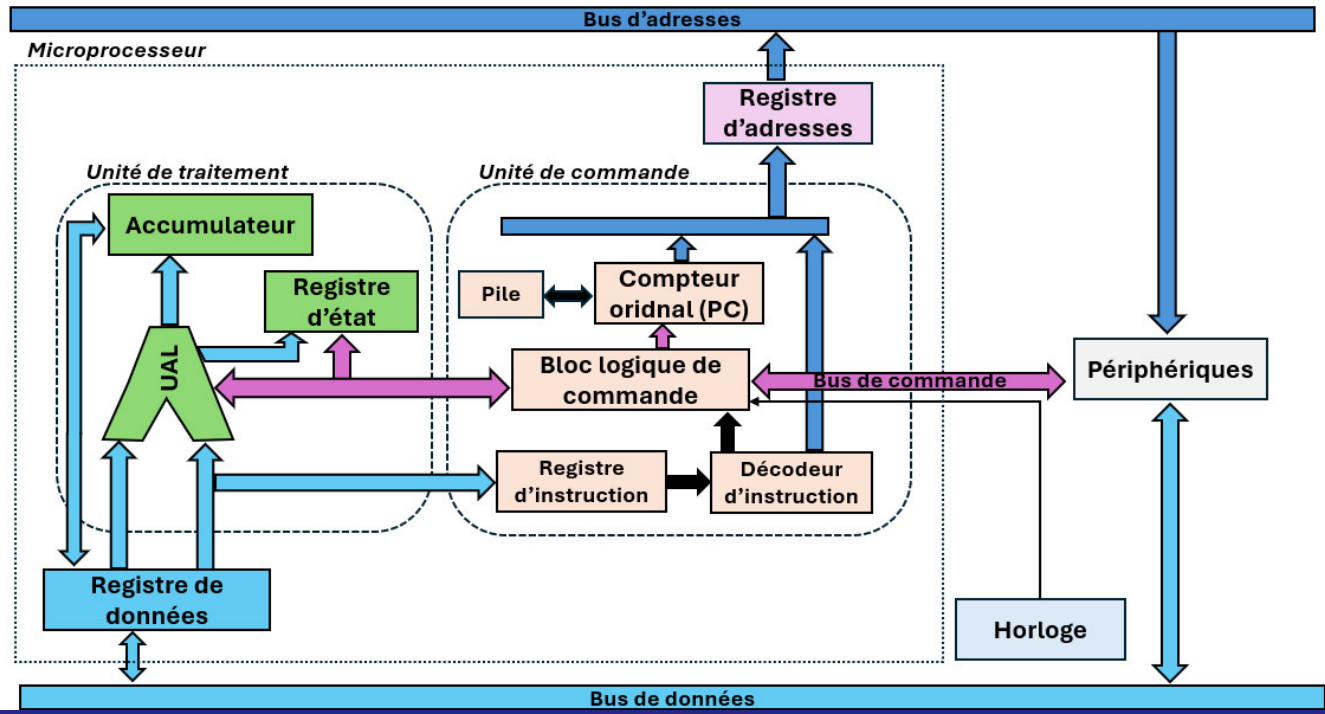
▷ Architecture de base d'un microprocesseur :

- Registre d'état est un registre dont chacun de ses bits est un indicateur dont l'état dépend du résultat de la dernière opération réalisée par l'UAL. On les appelle *indicateur d'état ou flag ou drapeaux*.
  Exemple : Retenue (carry : C), Zéro (Z), Signe (Sign : S), Parité (P)...
  Utilité :Dans un programme le résultat du test de leur état conditionne souvent le déroulement de la suite du programme.

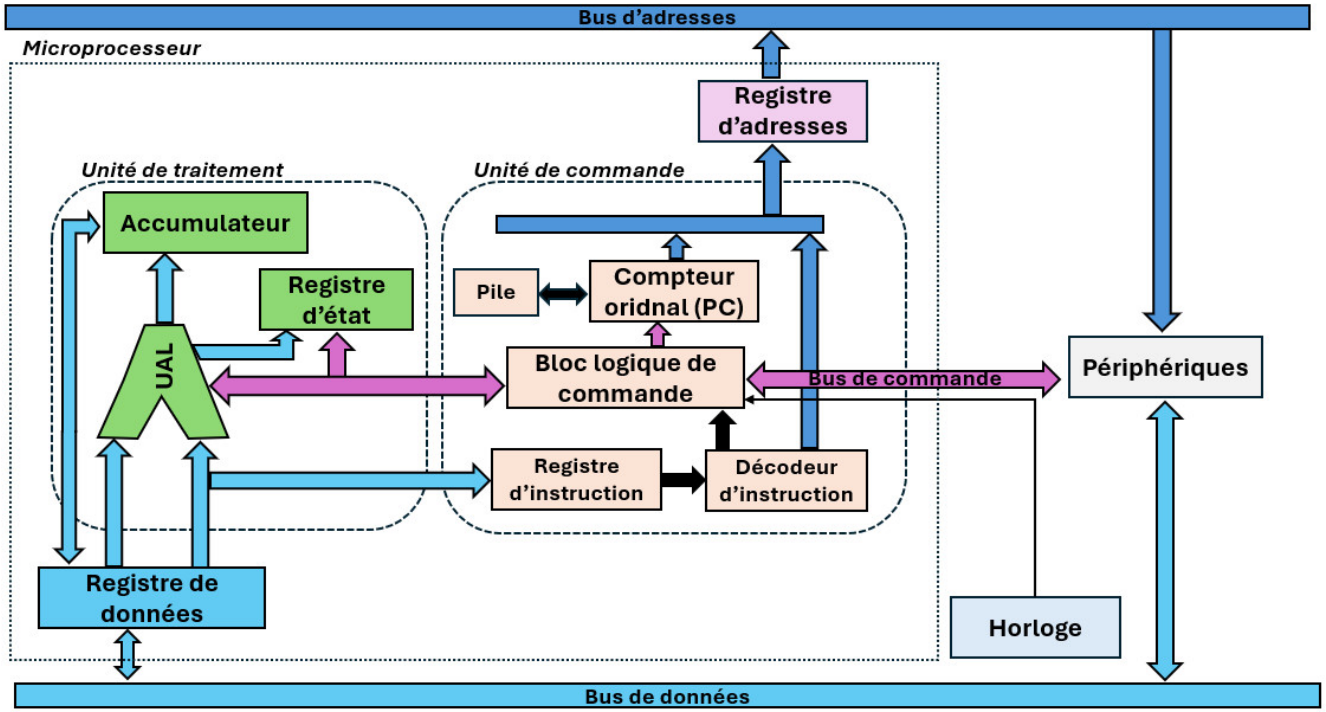


▷ **Architecture de base d'un microprocesseur** : L'Unité de commande Le rôle principale de cette unité est de séquencer le déroulement des instructions.

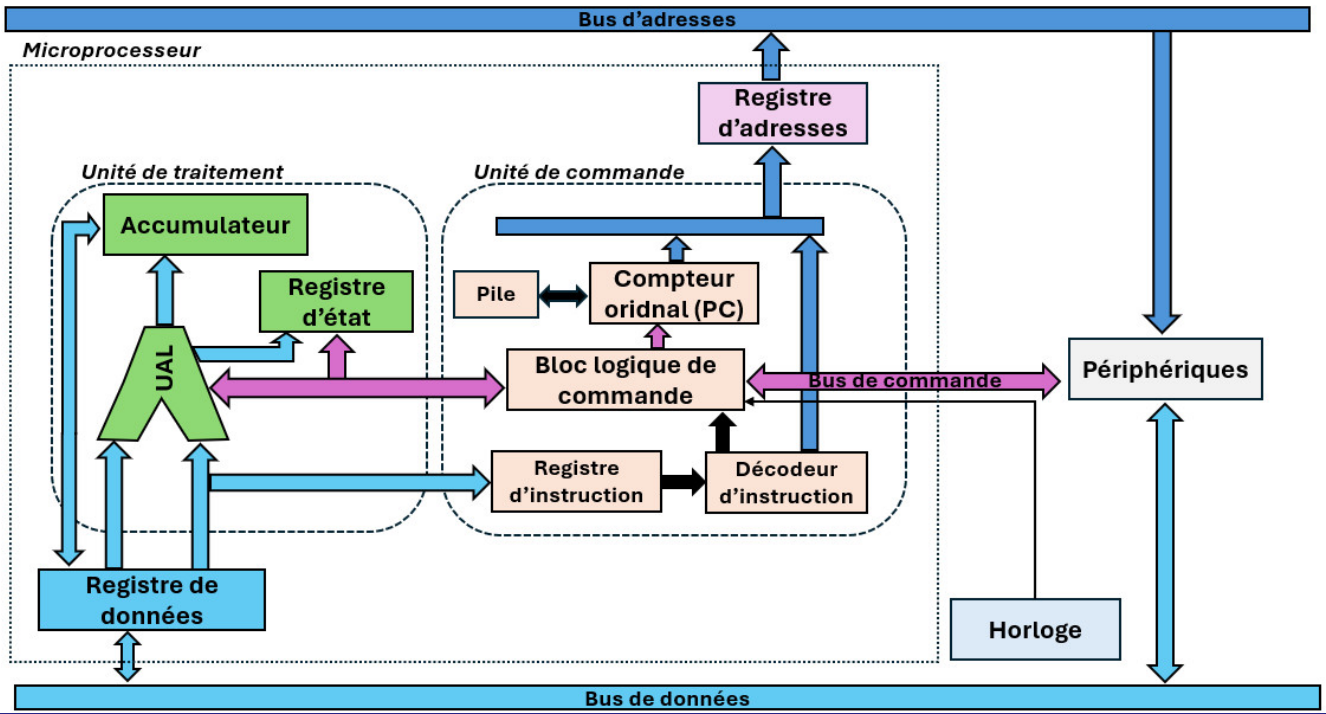
- En premier temps, elle se charge de rechercher l'instruction à exécuter en mémoire;
- puis, elle en assure le décodage pour déterminer la nature de la tâche à exécuter (exécution),
- ensuite, elle effectue la préparation de l'instruction suivante.



- **Architecture de base d'un microprocesseur** :
  - Compteur ordinal, aussi appelé pointeur d'instruction ou compteur de programme (en anglais Instruction Pointer ou Program Counter), est un registre spécial du microprocesseur dont le contenu est l'adresse mémoire de la prochaine instruction à exécuter.

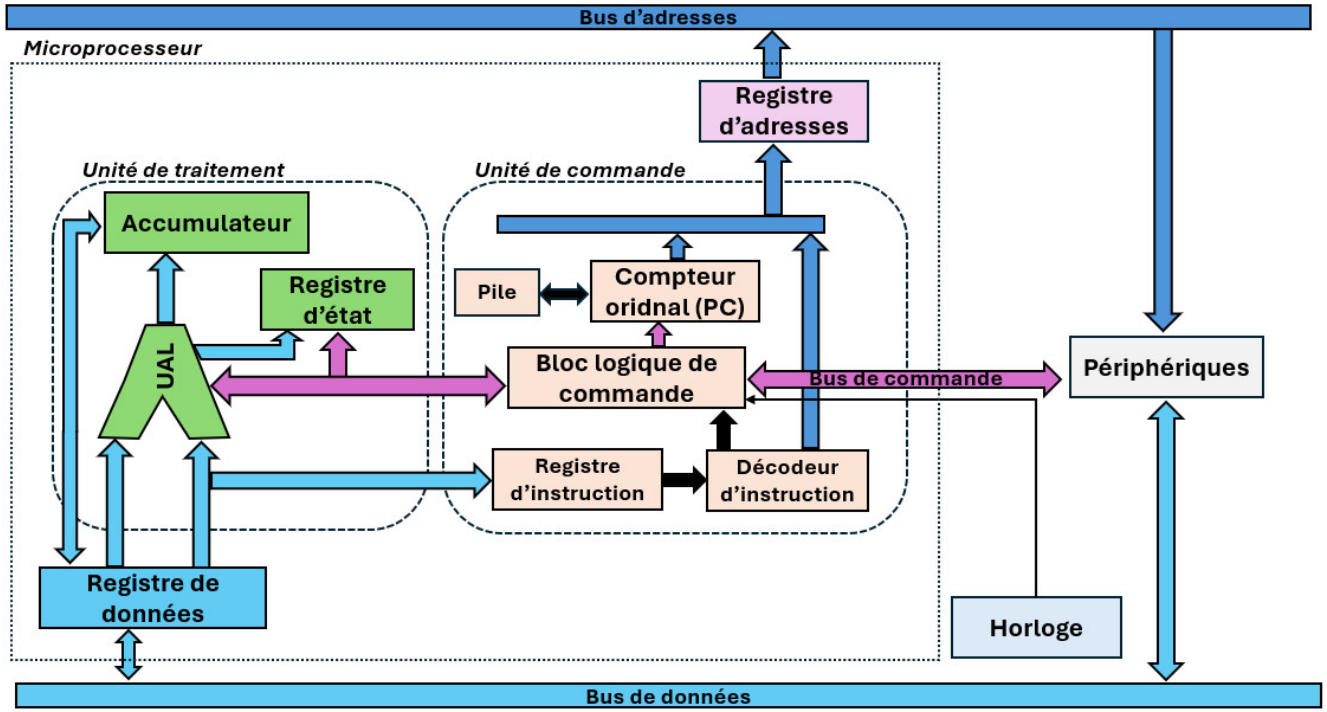


- **Architecture de base d'un microprocesseur** :
  - Pile: Le rôle principal de la pile est de sauvegarder le contenu de compteur ordinal, chaque fois qu'il y a un saut de programme afin par exemple d'appeler un sous-programme. Ce dernier est hébergé dans un emplacement de la mémoire. Quand son déroulement est terminé, la valeur initiale du compteur ordinal - sauvegardée dans la pile - réintégrera ce dernier, de façon à assurer la reprise du programme principal.

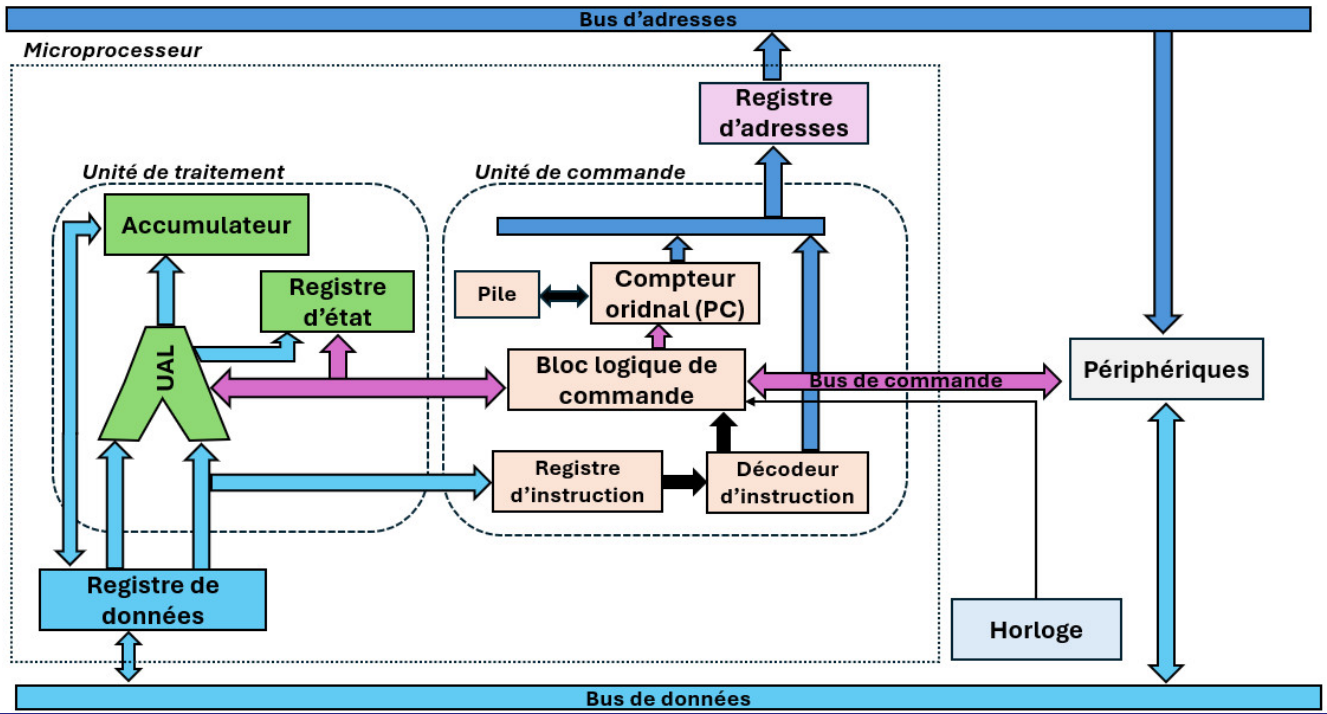


▷ Architecture de base d'un microprocesseur :

- Registre d'instruction et le décodeur d'instruction : chacune des instructions à exécuter est reçu par le registre instruction puis son décodage est effectué au niveau de décodeur d'instruction. On rappelle que chaque instruction est codée sous forme binaire.

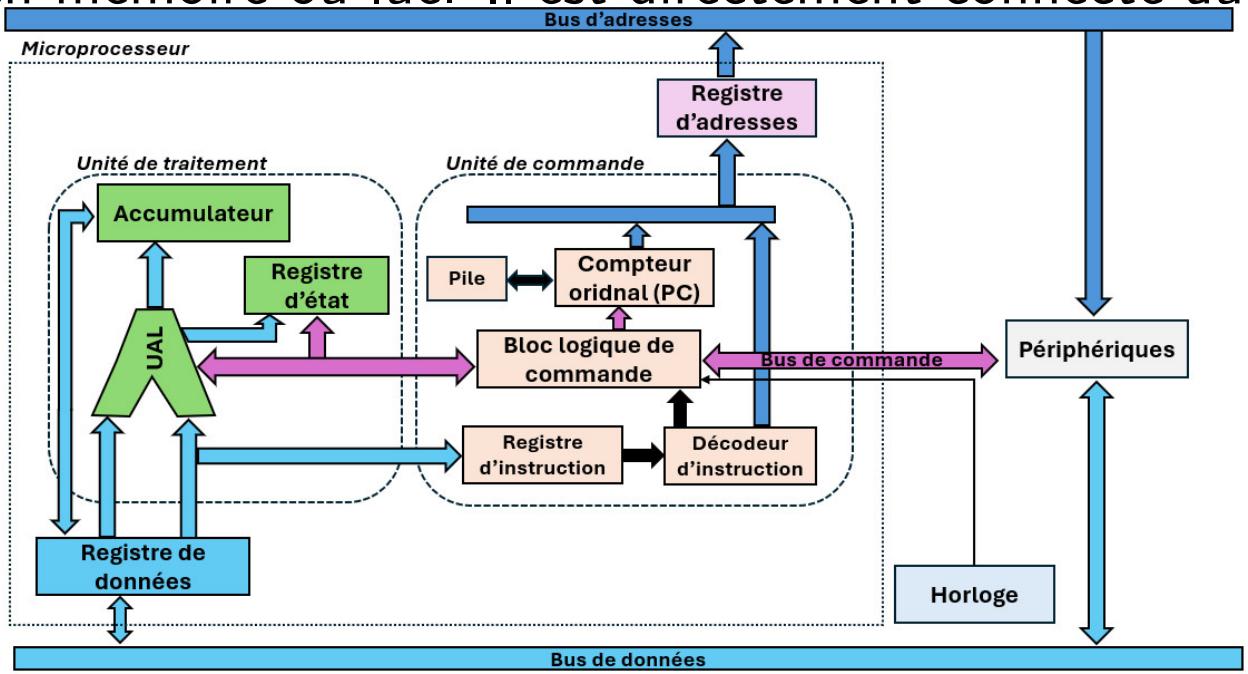


- **Architecture de base d'un microprocesseur** :
  - Bloc logique de commande (ou séquenceur) : son rôle est d'organiser l'exécution des instructions au rythme d'une horloge. Il génère tous les signaux de synchronisation internes/externes (bus de commande) du microprocesseur en fonction des différents signaux de commande provenant du registre d'état ou du décodeur d'instruction par exemple. C'est donc un automate réalisé soit de façon microprogrammée, on parle alors de micro-microprocesseur, soit de façon câblée.



▷ **Architecture de base d'un microprocesseur** : Registres d'adresses et données: servent à stocker temporairement des données, des instructions ou des adresses pendant l'exécution des programmes.

- Le registre d'adresses contient l'adresse servant à repérer l'emplacement mémoire que le microprocesseur désire utiliser. Au début d'un cycle de recherche et d'exécution, les contenus du registre d'adresses et du compteur ordinal sont donc identiques.
- Le registres de données appelé aussi registre tampon contient la donnée à écrire en mémoire ou lue. Il est directement connecté au bus de données.



▷ Fonctionnement d'un microprocesseur:

- Un microprocesseur sous tension est toujours prêt à exécuter les différentes instructions d'un programme qu'est déposé dans la mémoire, mais seulement si on lui en donne l'ordre en "levant un drapeau".
- Une instruction peut être codée sur un ou plusieurs octets. Le format générale d'une instruction est la suivante:

Opération	opérande 1	,	opérande 2

  Exemple:

  $\underbrace{\text{Additionner}}_{\text{opération}}, \underbrace{\text{donnée de case mémoire 1, donnée de la case mémoire 2}}_{\text{opérandes}}$

- L'exécution d'une instruction par le microprocesseur se fait en trois phases:
  - Récupération de l'instruction;
  - Décodage de l'instruction et recherche de l'opérande;
  - Exécution de l'instruction.

▷ Fonctionnement d'un microprocesseur:

- Un microprocesseur sous tension est toujours prêt à exécuter les différentes instructions d'un programme qu'est déposé dans la mémoire, mais seulement si on lui en donne l'ordre en "levant un drapeau".
- Une instruction peut être codée sur un ou plusieurs octets. Le format générale d'une instruction est la suivante:

Opération	opérande 1	,	opérande 2

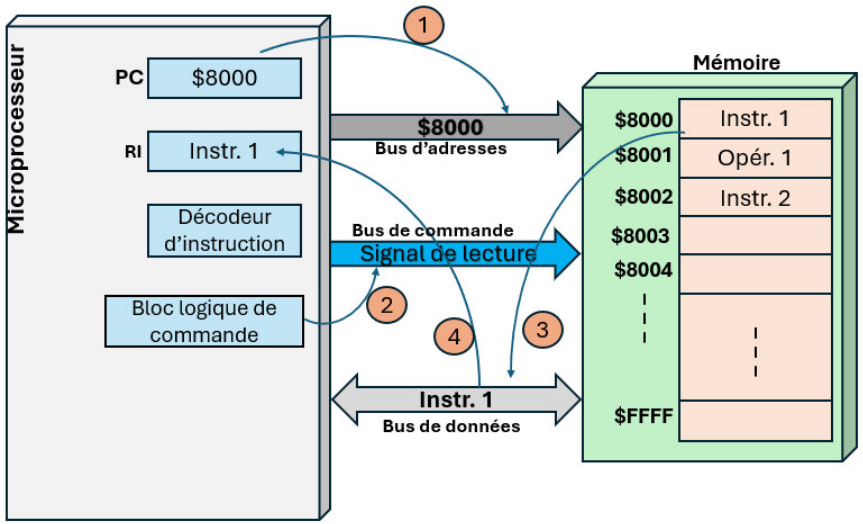
  Exemple:

  $\underbrace{\text{Additionner}}_{\text{opération}}, \underbrace{\text{donnée de case mémoire 1, donnée de la case mémoire 2}}_{\text{opérandes}}$

- L'exécution d'une instruction par le microprocesseur se fait en trois phases:
  - Récupération de l'instruction;
  - Décodage de l'instruction et recherche de l'opérande;
  - Exécution de l'instruction.

▷ Fonctionnement d'un microprocesseur: 1- Récupération de l'instruction

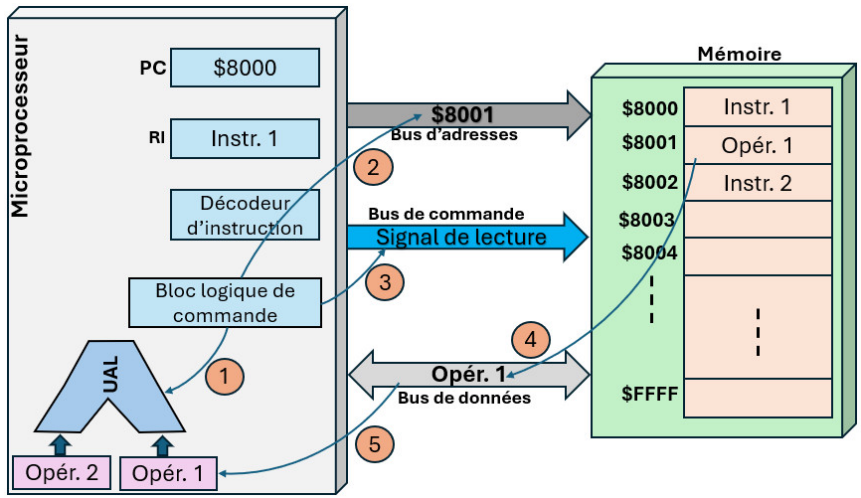
- L'adresse de l'instruction à exécuter se trouve toujours dans le compteur de programme.
- **Exemple** : supposons qu'il contienne la valeur $8000H. Ce nombre représente donc l'adresse de l'instruction à effectuer. Pour l'identifier, le µP va devoir en premier lieu aller la chercher (to fetch en anglais), ce qui implique toute une succession d'opérations élémentaires:
  - Transfert le contenu de compteur ordinal qui contient l'adresse de l'instruction suivante du programme vers le bus d'adresses;
  - Envoi d'un ordre de lecture à la mémoire centrale (Read : $\overline{RD}$);
  - Au bout d'un certain temps (temps d'accès à la mémoire), le contenu de la case mémoire sélectionnée est disponible sur le bus des données;
  - L'instruction est stockée dans le registre instruction du processeur.



▷ **Fonctionnement d'un microprocesseur**: 2- Décodage de l'instruction et recherche de l'opérande:

▷Le registre d'instruction contient maintenant le premier mot de l'instruction qui peut être codée sur plusieurs mots. Ce premier mot contient le code opératoire qui définit la nature de l'opération à effectuer et le nombre de mots de l'instruction. Le déroulement de la deuxième phase est comme suit:

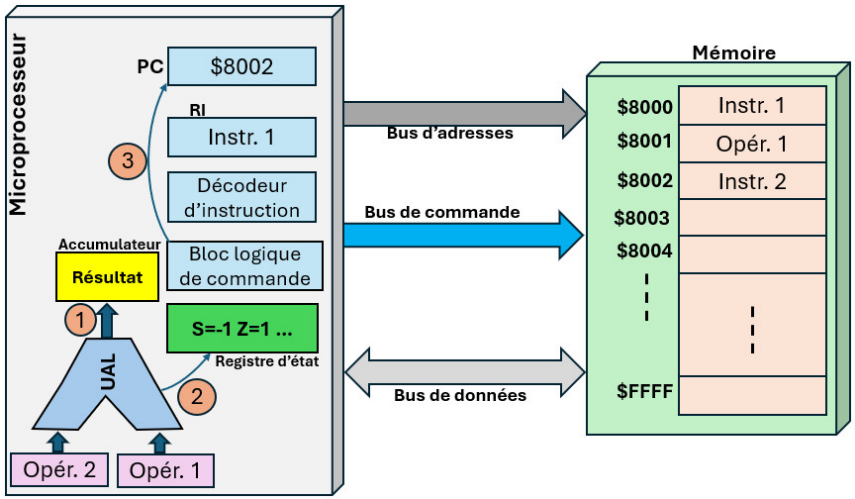
- Décodage:
  - Lecture éventuelle des autres mots d'instruction (selon le format);
  - L'unité de commande transforme l'instruction en une suite de commandes élémentaires nécessaires au traitement de l'instruction.
- Recherche d'opérandes. Si l'instruction à besoin des opérandes, alors ils peuvent être obtenu soit par :
  - Accès aux registres (si mode registre);
  - Calcul d'adresse et recherche d'opérandes mémoire (si mode mémoire), puis stocker l'opérande dans un registre.



Exemple : opérande 1 est stocké dans la mémoire par contre l'opérande 2 est stocké dans le registre.

▷ **Fonctionnement d'un microprocesseur**: 3- Exécution de l'instruction:

- Exécution. Deux types de l'exécution peuvent avoir lieu:
  - Calcul de l'adresse de branchement pour les instructions de contrôle
  - Calcul dans l'ALU pour les instructions arithmétiques ou logiques. Pour cela, le micro-programme réalisant l'instruction est exécuté.
- Les drapeaux sont positionnés (registre d'état).
- Écriture de résultats. Suivant le type d'exécution effectué, on distingue deux types d'écriture des résultats:
  - Modification de PC pour les instructions de contrôle;
  - Modification de l'opérande destination pour les instructions arithmétiques ou logiques;
- L'unité de commande positionne le PC pour l'instruction suivante.



Exemple : Exécution d'une instruction et stockage dans l'accumulateur.

▷ **Forme d'instructions: Architectures CISC et RISC**: En se basant sur le jeu d'instruction, on distingue entre deux grandes familles des microprocesseurs:

- Microprocesseur de type CISC (Complex Instruction Set Computer), on dit aussi microprocesseur à jeu d'instruction complexe;
  - **Exemple**: Motorola 6809, Intel 8086, Motorola 680x0;
- Microprocesseur de type RISC (Reduced Instruction Set Computer), on dit aussi microprocesseur à jeu d'instruction réduit.
  Exemple: SPARC, POWER, PowerPC, MIPS et ARM.

- **Forme d'instructions: Architectures CISC et RISC**:
  - Microprocesseur de type CISC (Complex Instruction Set Computer),
    - **Pourquoi CSIC**
      - Grand temps d'accès à la mémoire : vue que la mémoire travaillait lentement par rapport au processeur, on pensait qu'il était plus intéressant de soumettre au microprocesseur des instructions complexes.
      - Problèmes de conception des compilateurs : le développement des langages de haut niveau pose de nombreux problèmes quant à la conception de compilateurs. On a donc eu tendance à incorporer au niveau processeur des instructions plus proches de la structure de ces langages de haut niveau.
    - **Principe :** Pour ces raisons, l'architecture CISC a été développée où le microprocesseur doit exécuter des tâches complexes par un petit nombre d'instructions mais chacune nécessite un plus grand nombre de cycles d'horloge.
    - **Exemple:** ADD AX, [B200] ; additionne le contenu de la case mémoire qui se trouve à l'adresse B200 au contenu du registre AX.
      Dans ce cas, une seule instruction fait plusieurs choses : accéder à la mémoire, lire la donnée, puis l'additionner avec le contenu du registre AX.
    - Le code machine de ces instructions varie d'une instruction à l'autre et nécessite donc un décodeur complexe (micro-code).

- **Forme d'instructions: Architectures CISC et RISC**:
  - Microprocesseur de type RISC (Reduced Instruction Set Computer)
    - **Pourquoi RISC**
      - 80% des traitements des langages de haut niveau faisaient appel à seulement 20% des instructions du microprocesseur: Des études statistiques montrent que les programmes générés par les compilateurs se contentaient le plus souvent d'affectations, d'additions et de multiplications par des constantes. $\Longrightarrow$ D'où l'idée de réduire le jeu d'instructions et d'en améliorer la vitesse de traitement.
    - **Principe :**
      - Architecture RISC utilise un nombre réduit d'instructions (chargement, branchement, appel sous-programme).
      - Chacune de ces instructions s'exécutent ainsi en un seul cycle d'horloge.
      - Les accès à la mémoire s'effectuent seulement à partir de deux instructions (Load et Store).
    - **Exemple:**
      LOAD R1, 1234H ; charge le contenu de la case mémoire d'adresse 1234H dans le registre R1
      ADD R2, R1 ; additionne R2 avec R1. Dans ce cas, deux instructions sont utilisé pour faire l'addition : une pour charger, une pour additionner. Mais chaque instruction est très simple et rapide.

- **Forme d'instructions: Architectures CISC et RISC**:
  - Microprocesseur de type RISC (Reduced Instruction Set Computer)
    - **Pourquoi RISC**
      - 80% des traitements des langages de haut niveau faisaient appel à seulement 20% des instructions du microprocesseur: Des études statistiques montrent que les programmes générés par les compilateurs se contentaient le plus souvent d'affectations, d'additions et de multiplications par des constantes. $\Longrightarrow$ D'où l'idée de réduire le jeu d'instructions et d'en améliorer la vitesse de traitement.
    - **Principe :**
      - Architecture RISC utilise un nombre réduit d'instructions (chargement, branchement, appel sous-programme).
      - Chacune de ces instructions s'exécutent ainsi en un seul cycle d'horloge.
      - Les accès à la mémoire s'effectuent seulement à partir de deux instructions (Load et Store).
    - **Exemple:**
      LOAD R1, 1234H ; charge le contenu de la case mémoire d'adresse 1234H dans le registre R1
      ADD R2, R1 ; additionne R2 avec R1. Dans ce cas, deux instructions sont utilisé pour faire l'addition : une pour charger, une pour additionner. Mais chaque instruction est très simple et rapide.

▷ Comparaison entre CISC et RISC

CISC	RISC
- Instructions composés (complexes) sont exécutées en plusieurs cycles	- Instructions simples sont exécutées en un seul cycle
- Instructions ont un format variable	- Instructions ont un format fixe
- Divers instructions peuvent accéder à la mémoire	- Seulement les instruction load et store peuvent accéder à la mémoire
- Décodeur complexe (séquenceur micro-programmé)	- Décodeur simple (séquenceur câblé)
- Moins de registres	- Plus de registres
- Plus de modes d'adressage	- Moins de modes d'adressage
- Compilateur simple	- Compilateur complexe

Table: Comparaison entr CISC et RISC.

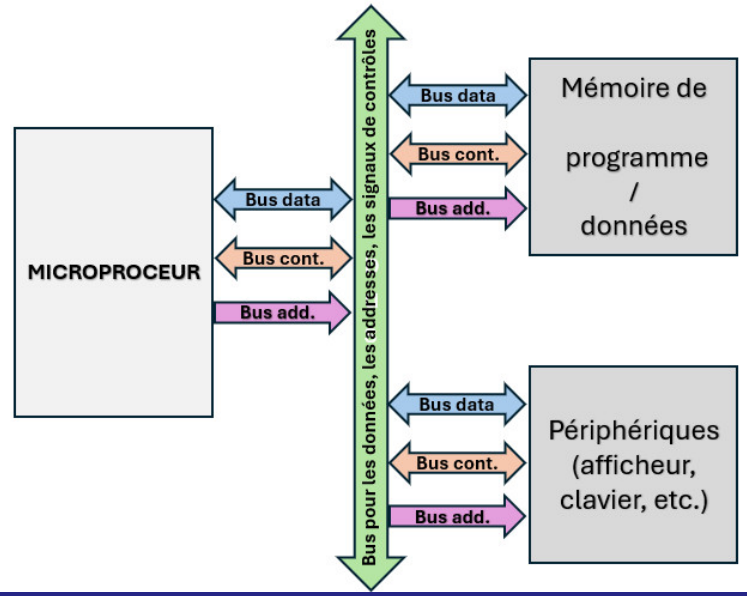
▷ **Procédure de transfert des données: Architectures de Von Neumann et Harvard**

La procédure de transfert des données et instructions entre le microprocesseur et les mémoires est un élément important qui conditionne directement les performances d'un microprocesseur. Il existe deux architectures d'accès à la mémoire par le microprocesseur:

- Von Neumann
- Harvard

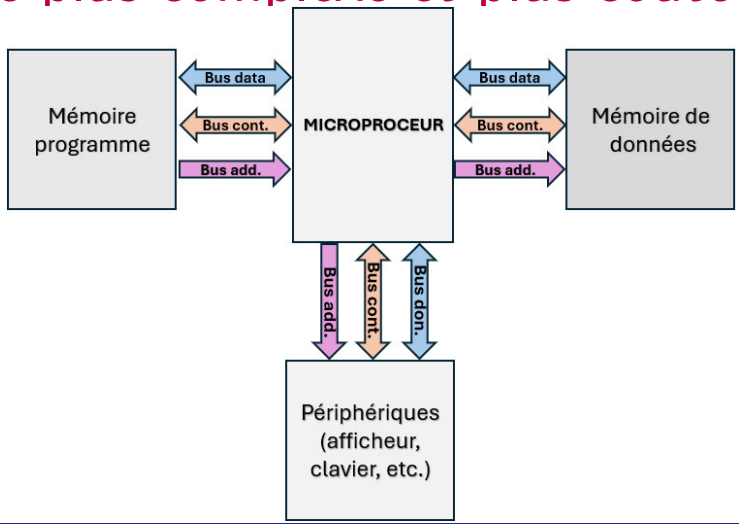
▷ Architectures de Von Neumann

- a été proposée en 1945 par le mathématicien et physicien Hongrois/Américain John Von Neumann.
- Son principe repose sur le concept de programme enregistré en mémoire (stored program concept)$\Longrightarrow$ les instructions du programme (code machine) et les données manipulées par ces instructions sont stockées à la fois dans le même espace mémoire adressé par le microprocesseur.
- Le microprocesseur communique donc avec cette mémoire au moyen d'un seul bus qui transporte à la fois les adresses, les données et les signaux de contrôles $\Longrightarrow$ Ce choix simplifie la conception matérielle et réduit les coûts
- leur inconvénient majeur est qu'il ne permet pas au processeur d'accéder simultanément à une instruction et à une donnée, ce qui limite la vitesse d'exécution$\Longrightarrow$ Cette limitation est connue sous le nom de Von Neumann bottleneck.



▷ Architectures de Harvard

- Propose de séparer entre la mémoire programme et la mémoire de données
- Chaque mémoire dispose de son propre bus d'adresses et de données, ce qui permet au processeur d'accéder simultanément à une instruction et à une donnée, améliorant ainsi significativement la vitesse d'exécution.
- Cette architecture permet donc de surmonter l'inconvénient de l'architecture de Von Neumann (Von Neumann bottleneck).
- Malgré que l'architecture Harvard permet d'améliorer la vitesse du microprocesseur, cependant, la séparation des mémoires programmes et de données rend la conception matérielle plus complexe et plus coûteuse.
- Elle est moins flexible que celle de Von Neumann. En effet, contrairement à l'architecture Harvard, Von Neumann permet d'utiliser la mémoire pour les données ou les programmes de manière interchangeable.



▷ Architectures Von Neumann vs. Harvard

Paramètres	Architecture Von Neumann	Architecture Harvard
- Mémoires	- Les données et les instructions (programme) sont stockées dans la même mémoire	- Les données et les instructions (programme) sont stockées dans des mémoires séparées
- Bus	- Bus d'adresse commun pour les données et les instructions	- Bus de données et d'instructions sont séparés
- Nombre de cycles	- Le programme est exécuté de façon séquentielle et nécessite plus de cycles	- Le programme est exécuté en parallèle avec les données, donc nécessite moins de cycles
- Transfert de données/instruction	- Données ou instruction transférés en un seul cycle	- Données et code transférés dans le même cycle
- Exigences matérielles	- Nécessite moins de matériel que Harvard	- Nécessite plus de matériel que Von Neumann

▷ Architectures Von Neumann vs Harvard : domaines d'applications

Architecture Von Neumann	Architecture Harvard
- Ordinateurs personnels (PC)	- Microcontrôleurs et microprocesseurs embarqués
- Serveurs	- Systèmes temps réel
- Supercalculateurs et ordinateurs utilisés en recherche scientifique	- Processeurs de traitement du signal (DSP)

Table: Domaines d'pplications des architectures Harvard et Von Neumann

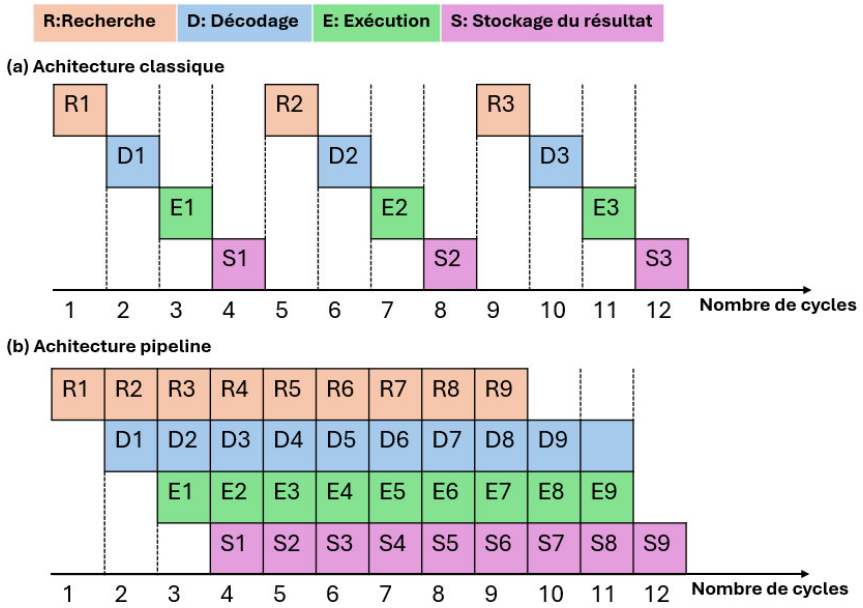
▷ Procédure d'exécution des instructions: Architectures classique et pipeline

Contexte: Plusieurs architectures ont été proposées pour améliorer le temps d'exécution des tâches par les microprocesseurs.

- certaines technologies proposent d'augmenter la fréquence de l'horloge du microprocesseur
  - l'inconvénient de cette technologie est que l'accélération des fréquences provoque un surcroît de consommation
  - ce qui entraîne le chauffage des composants électroniques.
- D'autres technologies proposent de diminuer le nombre moyen de cycles d'horloge nécessaire à l'exécution d'une instruction.
  - Parmi les différentes architectures proposées, on trouve l'architecture pipeline qui est la plus populaire dans les systèmes à microprocesseur.
  - Dans le cas d'une programmation en langage de haut niveau, cette amélioration peut se faire en optimisant le compilateur de telle sorte qu'il soit capable de sélectionner les séquences d'instructions minimisant le nombre moyen de cycles par instructions.

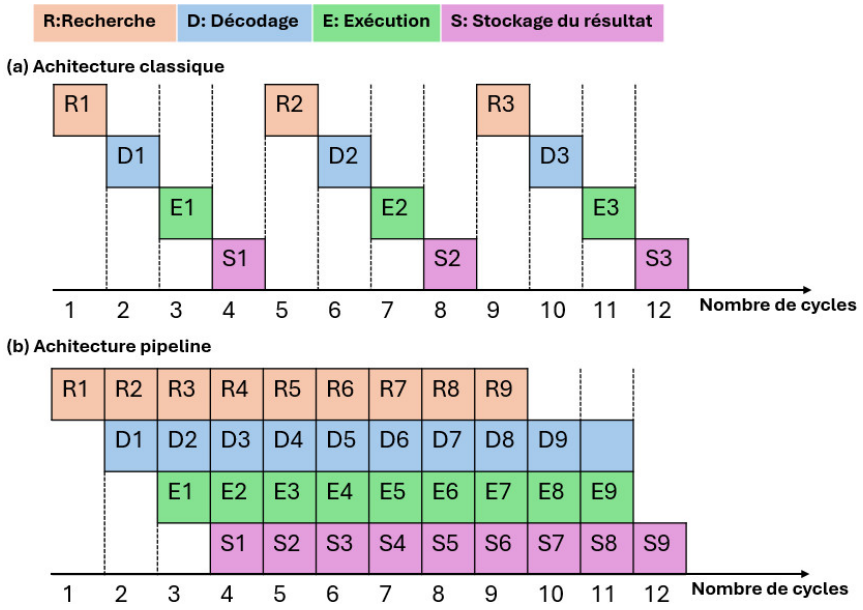
▷ Procédure d'exécution des instructions: Architectures classique et pipeline

- Dans le modèle classique, l'exécution est séquentielle
  - l'exécution de l'instruction suivante ne commence qu'après la fin de l'exécution de l'instruction encore. Le fonctionnement du microprocesseur n'est donc pas efficace.
- Dans le modèle pipeline, l'exécution est parallèle.
  - lorsque la première phase de l'exécution d'une instruction est achevée, l'instruction entre dans la seconde phase de son exécution et en parallèle la première phase de l'exécution de l'instruction suivante débute.
  - le microprocesseur débute l'exécution d'une instruction à chaque cycle et le pipeline est pleinement occupé à partir du quatrième cycle.
  - Le temps d'exécution d'une instruction n'est pas réduit, mais le débit d'exécution des instructions est considérablement augmenté.



▷ Procédure d'exécution des instructions: Architectures classique et pipeline

- Un microprocesseur à architecture pipeline se caractérise par le nombre d'étapes utilisées pour l'exécution d'une instruction. On appelle aussi ce nombre de phases le nombre d'étages du pipeline (est égal à 4 dans le cas de cet exemple).
- Ce nombre peut différer suivant le constructeur. Par exemple
  - Les Pentiums 2, 3 et 4 d'Intel comprennent respectivement un pipeline de 12, 10 et 20 étages
  - L'Athlon d'AMD comprend un pipeline de 11 étages.
- En plus de l'architecture pipeline d'autres architectures ont été développées comme: *+ Architecture superscalaire;+ Architecture pipeline et superscalaire; +Architecture VLIW; etc.*



▷ Architectures classique et pipeline: Gain de performance:

- **Définition**:Le gain de l'architecture pipeline est obtenu en calculant le rapport entre le nombre de cycles nécessaires pour exécuter un ensemble d'instructions en utilisant l'architecture classique sur le nombre de cycles nécessaires pour exécuter le même nombre d'instructions en utilisant l'architecture pipeline.
- **Exemple**, supposons qu'on a n instructions à exécuter et chaque instruction s'exécute en k cycles d'horloge, alors:
  - Avec l'architecture classique, il faut donc $n \times k$ cycles d'horloge;
  - Avec l'architecture pipeline, il faut k cycles d'horloge pour exécuter la première instruction puis $n-1$ cycles pour les $n-1$ instructions suivantes, c.à.d., au totale, on aura besoin de $k+n-1$ cycles.

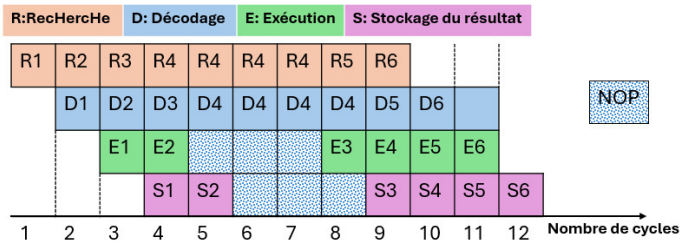
  Le gain est donc donné par:

$$G = \frac{n.k}{k+n-1} \tag{1}$$

- **Problèmes et solutions des pipelines**:
  - Problèmes rencontrés lors de l'exécution d'instructions par le pipeline sont classés en trois types:
    - **Aléas de Données (Data Hazards)** : proviennent lorsque une instruction dépend du résultat d'une instruction précédente non encore terminée.
      Exemple : ADD R1, R2, R3 suivie de SUB R4, R1, R5 (SUB dépend de R1).
    - **Aléas de Contrôle (Control Hazards)** : Surviennent des instructions de branchement (sauts, appels de fonctions) qui modifient le compteur de programme (PC). Dans ce cas, le processeur ne sait pas quelle instruction charger ensuite (branchement pris ou non) avant la fin du calcul, bloquant l'arrivée de nouvelles instructions.
    - **Aléas Structurels (Structural Hazards)** :proviennent quant deux instructions différentes dans le pipeline tentent d'utiliser la même ressource matérielle simultanément.
      Exemple : Un seul port d'accès mémoire utilisé à la fois pour chercher une instruction (Fetch) et écrire un résultat (Writeback).
  - Plusieurs solution ont été proposées pour résoudre les aléas d'un pipeline, y compris :
    - **Blocage (Stalling/Bubbles)** : Arrêter l'instruction fautive et celles qui

▷ Problèmes et solutions des pipelines:

- Plusieurs solution ont été proposées pour résoudre les aléas d'un pipeline, y compris :
  - **Blocage (Stalling/Bubbles)** : Arrêter l'instruction fautive et celles qui suivent jusqu'à la résolution du problème, insérant des cycles vides (NOP - No Operation).



  - **Prédiction de branchement :** Prédire l'issue d'un saut pour continuer à alimenter le pipeline, avec une pénalité si la prédiction est fausse.
  - **Réacheminement (Forwarding/Bypassing)** : Transférer directement le résultat produit à l'unité fonctionnelle qui en a besoin avant l'écriture dans le registre.

▷ **Définition**: Un ordinateur est un système électronique capable de stocker, traiter et restituer des informations. Son rôle fondamental est de recevoir des données en entrée, les traiter, puis de fournir un résultat en sortie, que ce soit sous forme visuelle (écran), imprimée (imprimante) ou via d'autres périphériques. Il est composé de deux partie principales :

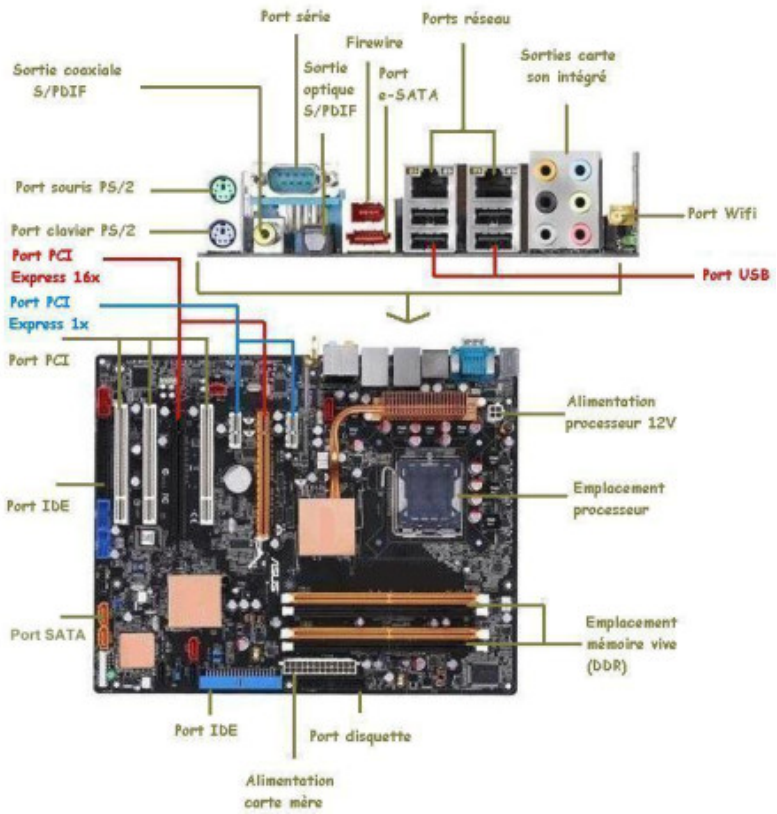
- **Partie matérielle (hardware)** : désigne l'ensemble des composants physiques et électroniques qui constituent la machine (comme la carte mère, les mémoires, etc.).
- **Partie logicielle (software)** : regroupe l'ensemble des programmes et applications qui exploitent les ressources matérielles de l'ordinateur (comme Windows, Office, etc.).



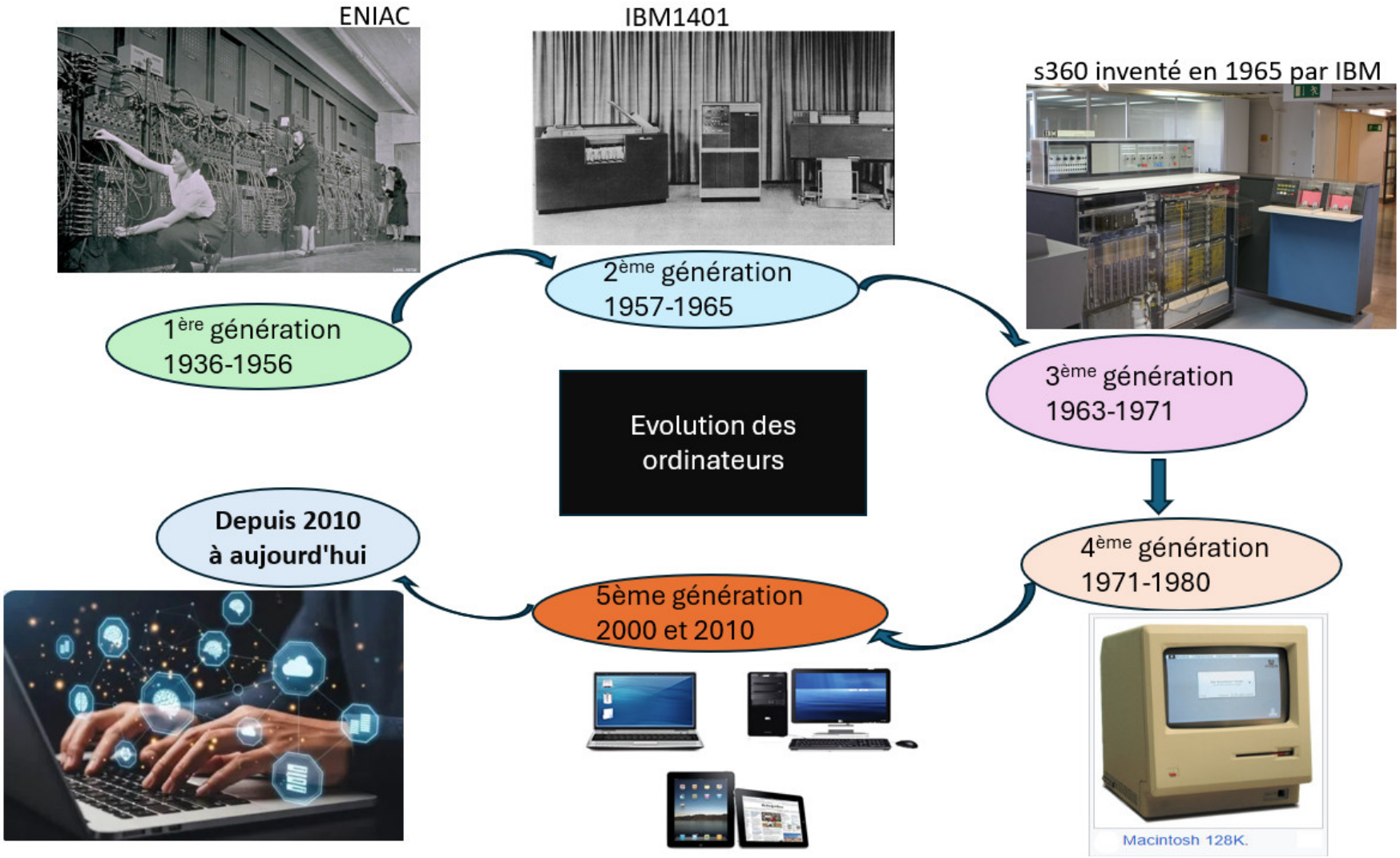


▷ **Carte mère et le microprocesseur**: La carte mère ou motherboard est un circuit imprimé qui relie et coordonne l'ensemble des éléments matériels (processeur, mémoires, périphériques).

- **Mémoire vive**, ou mémoire RAM (Random Access Memory)
- **Socket processeur** (CPU socket) : emplacement où est installé le microprocesseur.
- **BIOS/UEFI** : puce mémoire contenant le microprogramme de démarrage et les paramètres de configuration de la carte mère.
- **Bus d'extension et connecteurs PCIe** : permettent d'ajouter des cartes graphiques, cartes son, cartes réseau, etc.
- **Connecteurs de stockage** : ports **SATA** ou **NVMe** (M.2) pour disques durs, SSD et lecteurs optiques.
- **Connecteurs d'alimentation** : assurent l'apport en énergie depuis le bloc d'alimentation (24 broches ATX, connecteur CPU, PCIe).
- **Ports d'entrées/sorties (I/O)** : ils regroupent USB, HDMI/DisplayPort, audio, Ethernet, etc.
- **Régulateurs de tension (VRM)** : circuits assurant une alimentation stable du processeur et de la mémoire.



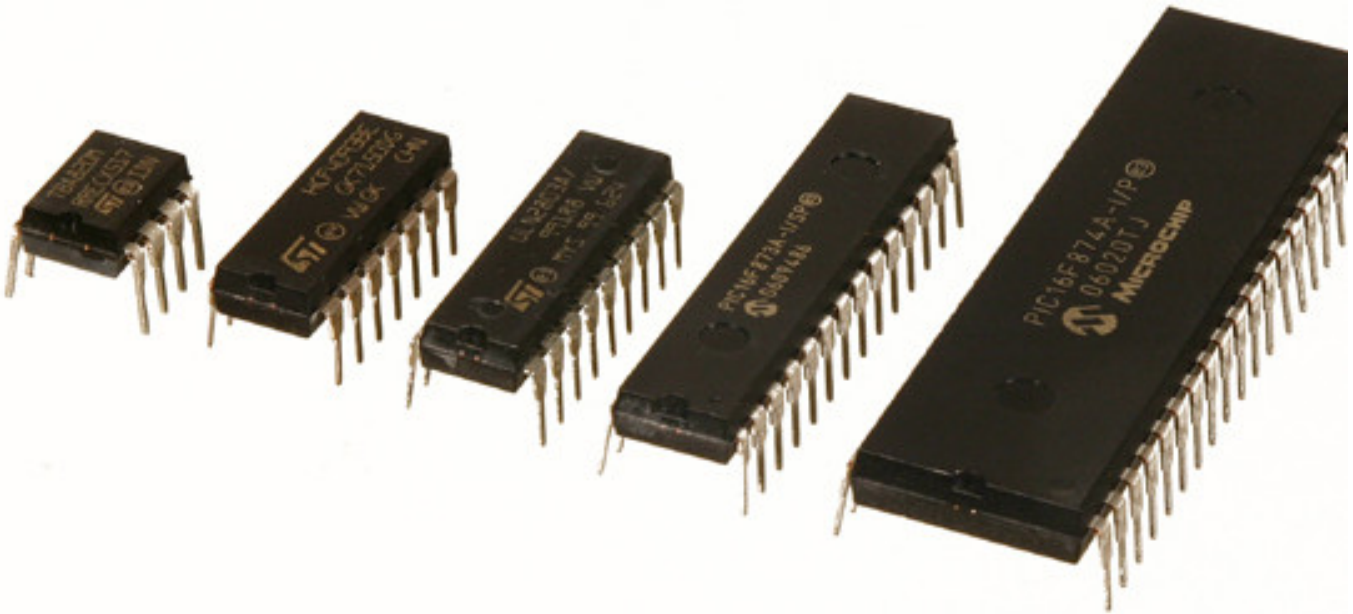
▷ Évolution des ordinateurs:



Exemples des systèmes à µp: Microcontrôleur

Un microcontrôleur est un circuit intégré regroupant tous les éléments d'un système à microprocesseur. Il contient généralement:

- Un microprocesseur;
- les mémoires de données et programmes (RAM, ROM, etc.);
- Des interfaces (synchrones ou asynchrones) pour le dialogue avec d'autres unités;
- Des convertisseurs analogiques/numériques pour la conversion des signaux;
- Des temporisateurs pour générer ou mesurer la durée des signaux; etc.



Thanks for your attention!

Feel free to ask any questions!